Jan 2026

Contents

1 Changes

1.1 .github/ISSUE_TEMPLATE/API.yml

```
name: API
description: Auto-filled issue for API building
title: "API: "
labels:
  - API
  - help wanted
  - Requirements needed
body:
  - type: textarea
    id: content
    attributes:
      label: Details
      value: |
        The details of this issue are not yet completed. The issue is here to track

        ## Key data structure

        What is the key data structure the API is built around.

        ## Need

        Why is this API needed, after it is completed, what will it make easier or a

        ## Requirements

        A tickbox list of the requirements of the API. This should include things wh

        ## Corresponding file system

        If it exists, the part of the PhysLean which corresponds to this API.

    validations:
```

```
      required: false
```

1.2 PhysLean.lean

```
import PhysLean.ClassicalMechanics.DampedHarmonicOscillator.Basic
import PhysLean.ClassicalMechanics.Lagrangian.TotalDerivativeEquivalence
import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
```

1.3 PhysLean/ClassicalMechanics/DampedHarmonicOscillator/Basic.

```
/-
Copyright (c) 2025 Nicola Bernini. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Nicola Bernini
-/
import PhysLean.Meta.Informal.SemiFormal
import PhysLean.ClassicalMechanics.EulerLagrange
import PhysLean.ClassicalMechanics.HamiltonsEquations
/-!

# The Damped Harmonic Oscillator

## i. Overview

The damped harmonic oscillator is a classical mechanical system correspondi
mass `m` under a restoring force `- k x` and a damping force `- γ ẋ`, where
spring constant, `γ` is the damping coefficient, `x` is the position, and `

The equation of motion for the damped harmonic oscillator is:
```
m ẍ + γ ẋ + k x = 0
```

Depending on the relationship between the damping coefficient and the natur
the system exhibits three different behaviors:
- **Underdamped** (γ² < 4mk) : Oscillatory motion with exponentially decayi
- **Critically damped** (γ² = 4mk) : Fastest return to equilibrium without
- **Overdamped** (γ² > 4mk) : Slow return to equilibrium without oscillatio

## ii. Key results

This module is currently a placeholder for future implementation. The follo
are planned to be formalized:
```

- `DampedHarmonicOscillator`: Structure containing the input data (mass, spring cons
  damping coefficient)
- `EquationOfMotion`: The equation of motion for the damped harmonic oscillator
- Solutions for underdamped, critically damped, and overdamped cases
- Energy dissipation properties
- Quality factor and relaxation time

iii. Table of contents

iv. References

References for the damped harmonic oscillator include:
- Landau & Lifshitz, Mechanics, page 76, section 25.
- Goldstein, Classical Mechanics, Chapter 2.

-/

```
namespace ClassicalMechanics
open Real
open Space
open InnerProductSpace

TODO "DHO01" "Define the DampedHarmonicOscillator structure with mass m, spring cons
  and damping coefficient γ."

TODO "DHO04" "Prove that energy is not conserved and derive the energy dissipation r

TODO "DHO05" "Derive solutions for the underdamped case (oscillatory with exponentia

TODO "DHO06" "Derive solutions for the critically damped case (fastest non-
oscillatory return)."

TODO "DHO07" "Derive solutions for the overdamped case (slow non-
oscillatory return)."
```

```
TODO "DHO08" "Define and prove properties of the quality factor Q."

TODO "DHO09" "Define and prove properties of the relaxation time τ."

TODO "DHO10" "Prove that the damped harmonic oscillator reduces to the unda

/-!

## A. The input data (placeholder)

The input data for the damped harmonic oscillator will consist of:
- Mass `m > 0`
- Spring constant `k > 0`
- Damping coefficient `γ ≥ 0`

-/

/-- Placeholder structure for the damped harmonic oscillator.
  The damped harmonic oscillator is specified by a mass `m`, a spring const
  and a damping coefficient `γ`. All parameters are assumed to be positive
negative
  for the damping coefficient). -/
structure DampedHarmonicOscillator where
  /-- The mass of the oscillator. -/
  m : ℝ
  /-- The spring constant of the oscillator. -/
  k : ℝ
  /-- The damping coefficient of the oscillator. -/
  γ : ℝ
  m_pos : 0 < m
  k_pos : 0 < k
  γ_nonneg : 0 ≤ γ

namespace DampedHarmonicOscillator

variable (S : DampedHarmonicOscillator)

@[simp]
lemma k_neq_zero : S.k ≠ 0 := Ne.symm (ne_of_lt S.k_pos)

@[simp]
lemma m_neq_zero : S.m ≠ 0 := Ne.symm (ne_of_lt S.m_pos)

/-!

## B. The natural angular frequency (placeholder)
```

```
The natural angular frequency ω₀ = √(k/m) will be defined here.

-/

/-- The natural (undamped) angular frequency of the oscillator, ω₀ = √(k/m). -
/
noncomputable def ω₀ : ℝ := √(S.k / S.m)

@[simp]
lemma ω₀_pos : 0 < S.ω₀ := sqrt_pos.mpr (div_pos S.k_pos S.m_pos)

lemma ω₀_sq : S.ω₀^2 = S.k / S.m := by
  rw [ω₀, sq_sqrt]
  exact div_nonneg (le_of_lt S.k_pos) (le_of_lt S.m_pos)

/-!
## C. Equation of motion (Tag: DHO03)

The damped harmonic oscillator with mass `m`, spring
constant `k`, and damping coefficient `γ` satisfies

    m ẍ + γ ẋ + k x = 0,

where `x : Time → ℝ` is the position as a function of time.
-/

/-- The equation of motion for the damped harmonic oscillator.

A function `x : Time → ℝ` is a solution if it satisfies

    S.m * x¨ + S.γ * ẋ + S.k * x = 0

for all times `t`. -/
noncomputable def EquationOfMotion (x : Time → ℝ) : Prop :=
  ∀ t : Time,
    S.m * (Time.deriv (Time.deriv x) t) +
    S.γ * (Time.deriv x t) +
    S.k * x t = 0

/-!
## D. The energies and energy dissipation (Tag: DHO04)

For the damped harmonic oscillator, the mechanical energy is

  E(t) = ½ S.m (ẋ(t))^2 + ½ S.k (x(t))^2,
```

```
where `x : Time → ℝ` is the position as a function of time.

If `x` satisfies the equation of motion

  S.m * ẍ + S.γ * ẋ + S.k * x = 0,

then differentiating `E` with respect to time and substituting the
equation of motion yields

  dE/dt = - S.γ * (ẋ(t))^2 ≤ 0

Thus the energy is non-increasing in time, and it is strictly decreasing
whenever `S.γ > 0` and `ẋ(t) ≠ 0`. In particular, for `S.γ > 0`
the energy is not conserved, and the energy dissipation rate is
proportional to the squared velocity.
-/

/-- The kinetic energy of the damped harmonic oscillator. -
/
noncomputable def kineticEnergy (x : Time → ℝ) : Time → ℝ :=
  fun t => (1 / 2 : ℝ) * S.m * (Time.deriv x t)^2

/-- The potential energy of the damped harmonic oscillator. -
/
noncomputable def potentialEnergy (x : Time → ℝ) : Time → ℝ :=
  fun t => (1 / 2 : ℝ) * S.k * (x t)^2

/-- Mechanical energy of the damped harmonic oscillator. -
/
noncomputable def energy (x : Time → ℝ) : Time → ℝ :=
  S.kineticEnergy x + S.potentialEnergy x

/-- Energy dissipation rate along a trajectory `x : Time → ℝ`.

  if `x` satisfies `S.equationOfMotion x`, then

  Time.deriv (S.energy x) t = - S.γ * (Time.deriv x t)^2,

so the energy is non-increasing and not conserved when `S.γ > 0`. -
/
noncomputable def energyDissipationRate (x : Time → ℝ) : Time → ℝ :=
  fun t => - S.γ * (Time.deriv x t)^2

/-!
```

```
## E. Damping regimes (placeholder)

The three damping regimes will be defined based on the discriminant γ² - 4mk.

-/

/-- The discriminant that determines the damping regime. -
/
noncomputable def discriminant : ℝ := S.γ^2 - 4 * S.m * S.k

/-- The system is underdamped when γ² < 4mk. -/
def IsUnderdamped : Prop := S.discriminant < 0

/-- The system is critically damped when γ² = 4mk. -/
def IsCriticallyDamped : Prop := S.discriminant = 0

/-- The system is overdamped when γ² > 4mk. -/
def IsOverdamped : Prop := S.discriminant > 0

end DampedHarmonicOscillator

end ClassicalMechanics
```

1.4 PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean

```
open Real Time ContDiff
to other specifications of initial conditions.

In this section, we implement alternative ways to specify initial conditions for the
oscillator. The standard `InitialConditions` type specifies position and velocity at
but in practice it is often useful to specify initial conditions at other times or i

Currently implemented:
- **Initial conditions at arbitrary time**: Specify position and velocity at any tim
  not necessarily at `t=0`.
  This is useful for problems where the natural reference time is not zero.

Future work (to be added in separate PRs) :
- Initial conditions from two positions at different times
- Initial conditions from two velocities at different times
- Amplitude-phase parametrization

All alternative forms can be converted to the standard `InitialConditions` type via
functions, and we prove that the converted initial conditions produce trajectories t
```

```
the original specifications.

-/

/-!

#### A.2.1. Initial conditions at arbitrary time

We define a type for initial conditions specified at an arbitrary time `t₀`
This is useful when the natural reference point for a problem is not at tim

-/

/-- Initial conditions for the harmonic oscillator specified at an arbitrar

  This structure allows specifying the position and velocity at any time `t
  at `t=0`. This is useful for problems where the natural reference time is

  The conditions can be converted to the standard `InitialConditions` forma
  using the `toInitialConditions` function. -/
structure InitialConditionsAtTime where
  /-- The time at which the initial conditions are specified. -
/
  t₀ : Time
  /-- The position at time t₀. -/
  x_t₀ : EuclideanSpace ℝ (Fin 1)
  /-- The velocity at time t₀. -/
  v_t₀ : EuclideanSpace ℝ (Fin 1)

/-!

##### A.2.1.1. Extensionality lemma

We prove an extensionality lemma for `InitialConditionsAtTime`.
@[ext]
lemma InitialConditionsAtTime.ext {IC₁ IC₂ : InitialConditionsAtTime}
    (h1 : IC₁.t₀ = IC₂.t₀) (h2 : IC₁.x_t₀ = IC₂.x_t₀) (h3 : IC₁.v_t₀ = IC₂.
    IC₁ = IC₂ := by
  cases IC₁
  cases IC₂
  simp_all

/-!

##### A.2.1.2. Conversion to standard initial conditions
```

```
We now define the conversion from `InitialConditionsAtTime` to the standard `Initial

The conversion works by "running the trajectory backward in time" from `t₀` to `0`.
Given that we know `x(t₀)` and `v(t₀)`, we use the harmonic oscillator solution form
with time-reversal to determine what `x(0)` and `v(0)` must have been.

Mathematically, if `x(t) = cos(ωt)·x₀ + (sin(ωt)/ω)·v₀`, then setting `t = t₀`:
  `x(t₀) = cos(ωt₀)·x₀ + (sin(ωt₀)/ω)·v₀`
  `v(t₀) = -ω·sin(ωt₀)·x₀ + cos(ωt₀)·v₀`

Solving this linear system for `x₀` and `v₀` gives the formulas below.

-/

namespace InitialConditionsAtTime

/-- Convert initial conditions at time `t₀` to standard initial conditions at `t=0`.

  This conversion uses the harmonic oscillator solution formula with time-
reversal.
  The resulting `InitialConditions` will produce a trajectory that passes through
  `x_t₀` with velocity `v_t₀` at time `t₀`.

  See `toInitialConditions_trajectory_at_t₀` and `toInitialConditions_velocity_at_t₀
  the correctness proofs. -/
noncomputable def toInitialConditions (S : HarmonicOscillator)
    (IC : InitialConditionsAtTime) : InitialConditions where
  x₀ := cos (S.ω * IC.t₀) • IC.x_t₀ - (sin (S.ω * IC.t₀) / S.ω) • IC.v_t₀
  v₀ := S.ω • sin (S.ω * IC.t₀) • IC.x_t₀ + cos (S.ω * IC.t₀) • IC.v_t₀

/-!
The correctness proofs showing that the conversion produces the expected trajectory
are given later in section D.1, after the trajectory machinery has been defined.
-/

TODO "6VZME" "Implement other initial conditions (deferred to future PRs). For examp
And convert them into the type `InitialConditions` above."

end InitialConditionsAtTime
lemma trajectories_unique (IC : InitialConditions) (x : Time → EuclideanSpace ℝ (Fin
    (hx : ContDiff ℝ ∞ x) :
    x = IC.trajectory S := by
  intro h
  rcases h with ⟨hEOM, hx0, hv0⟩

  -- Newton form for x
```

```
  have hNewt_x :
      ∀ t, S.m • ∂ₜ (∂ₜ x) t = force S (x t) :=
    (S.equationOfMotion_iff_newtons_2nd_law (xₜ := x) hx).1 hEOM

  -- Newton form for the explicit trajectory
  have hTrajContDiff : ContDiff ℝ ∞ (IC.trajectory S) := by
    -- trajectory_contDiff already exists and is [fun_prop]
    fun_prop

  have hNewt_traj :
      ∀ t, S.m • ∂ₜ (∂ₜ (IC.trajectory S)) t = force S ((IC.trajectory S) t
    (S.equationOfMotion_iff_newtons_2nd_law (xₜ := IC.trajectory S) hTrajCo
      (trajectory_equationOfMotion S IC)

  -- Define the difference y = x - traj
  set y : Time → EuclideanSpace ℝ (Fin 1) := fun t => x t - IC.trajectory S

  have hyContDiff : ContDiff ℝ ∞ y := by
    -- ContDiff closed under subtraction
    simpa [hydef] using hx.sub hTrajContDiff

  -- First derivative of y
  have hy_deriv : ∂ₜ y = fun t => ∂ₜ x t - ∂ₜ (IC.trajectory S) t := by
    funext t
    -- same style as in trajectory_velocity: unfold Time.deriv and use fder
    rw [hydef, Time.deriv]
    -- ContDiff implies DifferentiableAt - use this explicitly since fun_pr
    -- ContDiff ℝ ∞ f implies ContDiffAt ℝ ∞ f t for any t
    have hx_contDiffAt : ContDiffAt ℝ ∞ x t := hx.contDiffAt
    have htraj_contDiffAt : ContDiffAt ℝ ∞ (IC.trajectory S) t := hTrajCont
    have hx_diff : DifferentiableAt ℝ x t :=
      ContDiffAt.differentiableAt hx_contDiffAt (by norm_num : (1 : ℕ∞) ≤ ∞
    have htraj_diff : DifferentiableAt ℝ (IC.trajectory S) t :=
      ContDiffAt.differentiableAt htraj_contDiffAt (by norm_num : (1 : ℕ∞)
    rw [fderiv_fun_sub hx_diff htraj_diff]
    simp only [ContinuousLinearMap.sub_apply, Time.deriv]

  -- Second derivative of y
  have hy_deriv2 :
      ∂ₜ (∂ₜ y) = fun t => ∂ₜ (∂ₜ x) t - ∂ₜ (∂ₜ (IC.trajectory S)) t := by
    funext t
    rw [hy_deriv, Time.deriv]
    -- now differentiate (∂ₜ x - ∂ₜ traj)
    -- use differentiability of time-derivatives from ContDiff
    have hx1 : Differentiable ℝ (fun t => ∂ₜ x t) :=
      deriv_differentiable_of_contDiff x hx
```

```
    have htr1 : Differentiable ℝ (fun t => ∂ₜ (IC.trajectory S) t) :=
      deriv_differentiable_of_contDiff (IC.trajectory S) hTrajContDiff
    -- Apply fderiv_fun_sub and use Time.deriv to convert back
    -- Differentiable ℝ f means ∀ x, DifferentiableAt ℝ f x
    -- In Mathlib, Differentiable is defined as ∀ x, DifferentiableAt, so we can app
    have hx1_at : DifferentiableAt ℝ (fun t => ∂ₜ x t) t := hx1 t
    have htr1_at : DifferentiableAt ℝ (fun t => ∂ₜ (IC.trajectory S) t) t := htr1 t
    rw [fderiv_fun_sub hx1_at htr1_at]
    -- Now we need to show fderiv of (fun t => fderiv ℝ x t 1) equals fderiv of (∂ₜ
    -- This follows from Time.deriv f t = fderiv ℝ f t 1
    simp only [ContinuousLinearMap.sub_apply]
    rw [Time.deriv, Time.deriv]

  -- Newton form for y (linearity of force)
  have hNewt_y : ∀ t, S.m • ∂ₜ (∂ₜ y) t = force S (y t) := by
    intro t
    have hy2t : ∂ₜ (∂ₜ y) t =
        (∂ₜ (∂ₜ x) t - ∂ₜ (∂ₜ (IC.trajectory S)) t) := by
      simpa using congrFun hy_deriv2 t

    -- Expand and substitute Newton laws for x and traj, then fold back using force_
    calc
      S.m • ∂ₜ (∂ₜ y) t
          = S.m • (∂ₜ (∂ₜ x) t - ∂ₜ (∂ₜ (IC.trajectory S)) t) := by
              simp [hy2t]
      _ = (S.m • ∂ₜ (∂ₜ x) t) - (S.m • ∂ₜ (∂ₜ (IC.trajectory S)) t) := by
              simp [smul_sub]
      _ = force S (x t) - force S ((IC.trajectory S) t) := by
              simp [hNewt_x t, hNewt_traj t]
      _ = force S (y t) := by
              -- force = -k•x, so it is linear: force(x) - force(traj) = force(x-
traj)
              -- and y t = x t - traj t by definition
              simp [hydef, force_eq_linear, smul_sub]

  -- Turn Newton form back into EquationOfMotion for y
  have hEOM_y : S.EquationOfMotion y :=
    (S.equationOfMotion_iff_newtons_2nd_law (xₜ := y) hyContDiff).2 hNewt_y

  -- Initial conditions for y are zero
  have hy0 : y 0 = 0 := by
    -- y 0 = x 0 - traj 0 = IC.x₀ - IC.x₀
    simp [hydef, hx0]

  have hyv0 : ∂ₜ y 0 = 0 := by
    -- ∂ₜ y 0 = ∂ₜ x 0 - ∂ₜ traj 0 = IC.v₀ - IC.v₀
```

```
    rw [congr_fun hy_deriv 0]
    rw [hv0, trajectory_velocity_at_zero S IC]
    simp

  -- Energy at time 0 is 0
  have hE0 : S.energy y 0 = 0 := by
    -- unfold energy, kinetic, potential and use hy0, hyv0
    simp [HarmonicOscillator.energy, HarmonicOscillator.kineticEnergy,
      HarmonicOscillator.potentialEnergy, hy0, hyv0, one_div, smul_eq_mul]

  -- Energy is constant, hence always 0
  have hE : ∀ t, S.energy y t = 0 := by
    intro t
    have ht := S.energy_conservation_of_equationOfMotion' (xₜ := y) hyContD
    simpa [hE0] using ht

  -- From energy=0 and positivity => y(t)=0
  have hy_all : ∀ t, y t = 0 := by
    intro t
    have hEt : S.energy y t = 0 := hE t

    have hk_nonneg : 0 ≤ S.kineticEnergy y t := by
      unfold HarmonicOscillator.kineticEnergy
      have hcoeff : 0 ≤ (1 / (2 : ℝ)) * S.m := by
        exact mul_nonneg (by norm_num) (le_of_lt S.m_pos)
      -- Use the same approach as for potential energy below
      have hin : 0 ≤ inner ℝ (∂ₜ y t) (∂ₜ y t) := by
        -- For EuclideanSpace ℝ (Fin 1), inner product with itself is nonne
        exact real_inner_self_nonneg (x := ∂ₜ y t)
      exact mul_nonneg hcoeff hin

    have hp_nonneg : 0 ≤ S.potentialEnergy (y t) := by
      unfold HarmonicOscillator.potentialEnergy
      -- potentialEnergy = (1/2) * k * ⟪y,y⟫
      simp only [one_div, smul_eq_mul]
      -- Goal is 0 ≤ 2⁻¹ * (S.k * inner ℝ (y t) (y t))
      apply mul_nonneg
      · norm_num -- 0 ≤ 2⁻¹
      · -- 0 ≤ S.k * inner ℝ (y t) (y t)
        have hk_pos : 0 ≤ S.k := le_of_lt S.k_pos
        have hin : 0 ≤ inner ℝ (y t) (y t) := by
          -- For EuclideanSpace ℝ (Fin 1), inner product with itself is non
          exact real_inner_self_nonneg (x := y t)
        exact mul_nonneg hk_pos hin

    have hp_le : S.potentialEnergy (y t) ≤ S.energy y t := by
```

```
      unfold HarmonicOscillator.energy
      exact le_add_of_nonneg_left hk_nonneg

    have hp0 : S.potentialEnergy (y t) = 0 := by
      have : S.potentialEnergy (y t) ≤ 0 := by
        calc
          S.potentialEnergy (y t) ≤ S.energy y t := hp_le
          _ = 0 := hEt
      exact le_antisymm this hp_nonneg

    -- extract ⟪y,y⟫ = 0 from potentialEnergy = 0, then y=0
    have hy_inner0 : inner ℝ (y t) (y t) = 0 := by
      -- potentialEnergy = (1/2) * k * ⟪y,y⟫
      have hmul : ((1 / (2 : ℝ)) * S.k) * inner ℝ (y t) (y t) = 0 := by
        simpa [HarmonicOscillator.potentialEnergy, one_div, smul_eq_mul, mul_assoc]
      have hcoeff : ((1 / (2 : ℝ)) * S.k) ≠ 0 := by
        exact mul_ne_zero (by norm_num) (S.k_neq_zero)
      rcases mul_eq_zero.mp hmul with hcoeff0 | hinner
      · exact (False.elim (hcoeff hcoeff0))
      · exact hinner

    exact (inner_self_eq_zero.mp hy_inner0)

  -- Conclude x = traj
  funext t
  have : y t = 0 := hy_all t
  -- y t = x t - traj t
  simpa [hydef] using (sub_eq_zero.mp this)
end InitialConditions

/-!

## D.1. Correctness of InitialConditionsAtTime conversion

We now prove the correctness lemmas for the `InitialConditionsAtTime.toInitialCondit
conversion function. These show that the conversion produces a trajectory that passe
the specified position and velocity at the specified time.

-/

namespace InitialConditionsAtTime

/-- The trajectory resulting from `toInitialConditions` passes through the specified
  position `x_t₀` at time `t₀`. -/
@[simp]
lemma toInitialConditions_trajectory_at_t₀ (S : HarmonicOscillator)
```

```
    (IC : InitialConditionsAtTime) :
    (IC.toInitialConditions S).trajectory S IC.t₀ = IC.x_t₀ := by
  rw [InitialConditions.trajectory_eq, toInitialConditions]
  ext i
  simp only [PiLp.add_apply, PiLp.smul_apply, PiLp.sub_apply, smul_eq_mul]
  have h1 : cos (S.ω * IC.t₀.val) ^ 2 + sin (S.ω * IC.t₀.val) ^ 2 = 1 :=
    cos_sq_add_sin_sq (S.ω * IC.t₀.val)
  field_simp [S.ω_neq_zero]
  linear_combination S.ω * IC.x_t₀.ofLp i * h1

/-- The trajectory resulting from `toInitialConditions` has the specified
  velocity `v_t₀` at time `t₀`. -/
@[simp]
lemma toInitialConditions_velocity_at_t₀ (S : HarmonicOscillator)
    (IC : InitialConditionsAtTime) :
    ∂ₜ ((IC.toInitialConditions S).trajectory S) IC.t₀ = IC.v_t₀ := by
  rw [InitialConditions.trajectory_velocity, toInitialConditions]
  ext i
  simp only [PiLp.add_apply, PiLp.smul_apply, PiLp.sub_apply, smul_eq_mul,
  have h1 : cos (S.ω * IC.t₀.val) ^ 2 + sin (S.ω * IC.t₀.val) ^ 2 = 1 :=
    cos_sq_add_sin_sq (S.ω * IC.t₀.val)
  field_simp [S.ω_neq_zero]
  linear_combination IC.v_t₀.ofLp i * h1

/-- The energy of the trajectory at time `t₀` equals the energy computed fr
  initial conditions at `t₀`. -/
lemma toInitialConditions_energy_at_t₀ (S : HarmonicOscillator)
    (IC : InitialConditionsAtTime) :
    S.energy ((IC.toInitialConditions S).trajectory S) IC.t₀ =
    1/2 * (S.m * ‖IC.v_t₀‖^2 + S.k * ‖IC.x_t₀‖^2) := by
  unfold energy kineticEnergy potentialEnergy
  simp only [toInitialConditions_trajectory_at_t₀, toInitialConditions_velo
  rw [real_inner_self_eq_norm_sq, real_inner_self_eq_norm_sq]
  simp only [smul_eq_mul]
  ring

end InitialConditionsAtTime

namespace InitialConditions
```

1.5 PhysLean/ClassicalMechanics/Lagrangian/TotalDerivativeEquiv

```
/-
```

Authors: Rein Zustand
-/
import PhysLean.Mathematics.VariationalCalculus.HasVarGradient
/-!

Equivalent Lagrangians under Total Derivatives

i. Overview

Two Lagrangians are physically equivalent if they differ by a total time derivative
d/dt F(q, t). This is because the Euler-Lagrange equations depend only on extremizin
the action integral, and total derivatives don't affect which paths are extremal.

This module defines the key concept of a function being a total time derivative,
which is essential for analyzing symmetries like Galilean invariance.

Note: Some authors call this "gauge equivalence" by analogy with gauge transformatio
in field theory, but we avoid that terminology here since no gauge fields are involv

ii. Key insight

A general function δL(r, v, t) is a total time derivative if there exists a function
F(r, t) (independent of velocity) such that:
  δL(r, v, t) = d/dt F(r, t) = fderiv ℝ F (r, t) (v, 1)

By the chain rule, this expands to:
  δL(r, v, t) = ∂F/∂t + ⟨∇ᵣF, v⟩

For the special case where δL depends only on velocity v (not position or time),
this implies a strong constraint:
  δL(v) = ⟨g, v⟩ for some constant vector g

This is because:
1. d/dt F(r, t) = ∂F/∂t + ⟨∇F, v⟩
2. For δL to be r-independent, ∇F must be r-independent
3. For δL to be t-independent, the time-dependent part must vanish
4. The result is δL = ⟨g, v⟩ for constant g

iii. Key definitions

- `IsTotalTimeDerivative`: General case for δL(r, v, t)
- `IsTotalTimeDerivativeVelocity`: Velocity-only case, equivalent to δL(v) = ⟨g, v⟩

iv. References

```
- Landau & Lifshitz, "Mechanics", §2 (The principle of least action)
- Landau & Lifshitz, "Mechanics", §4 (The Lagrangian for a free particle)

-/

namespace ClassicalMechanics

open InnerProductSpace

namespace Lagrangian

/-!

## A. General Total Time Derivative

-/

/-- A function δL(r, v, t) is a total time derivative if it can be written
    for some function F that depends on position and time but not velocity.

    Mathematically: δL(r, v, t) = fderiv ℝ F (r, t) (v, 1)

    By the chain rule, this equals ∂F/∂t(r, t) + ⟨∇ᵣF(r, t), v⟩.

    This is the most general form of Lagrangian equivalence under total der
    The key point is that F must be independent of velocity. -
/
def IsTotalTimeDerivative {n : ℕ}
    (δL : EuclideanSpace ℝ (Fin n) → EuclideanSpace ℝ (Fin n) → ℝ → ℝ) : Pr
  ∃ (F : EuclideanSpace ℝ (Fin n) × ℝ → ℝ) (_ : Differentiable ℝ F),
    ∀ r v t, δL r v t = fderiv ℝ F (r, t) (v, 1)

/-!

## B. Velocity-Only Total Time Derivative

When δL depends only on velocity (the free particle case), the condition si

-/

/-- A velocity-only function that is a total time derivative must be linear

    If δL depends only on velocity and equals d/dt F(r, t) for some F,
    then δL(v) = ⟨g, v⟩ for some constant vector g.
```

```
    This characterization comes from the requirement that:
    - d/dt F(r, t) = ∂F/∂t + ⟨∇F, ṙ⟩ = ∂F/∂t + ⟨∇F, v⟩
    - For the result to be independent of r and t, we need ∇F = g (constant) and ∂F/
    - Thus δL(v) = ⟨g, v⟩

    WLOG, we assume `δL 0 = 0` since constants are total derivatives (c = d/dt(c·t))
    and can be absorbed without affecting the equations of motion. -
/
lemma isTotalTimeDerivativeVelocity {n : ℕ}
    (δL : EuclideanSpace ℝ (Fin n) → ℝ)
    (hδL0 : δL 0 = 0)
    (h : IsTotalTimeDerivative (fun _ v _ => δL v)) :
    ∃ g : EuclideanSpace ℝ (Fin n), ∀ v, δL v = ⟪g, v⟫_ℝ := by
  classical
  rcases h with ⟨F, hFdiff, hEq⟩

  -- Derivative of F at (0,0)
  let dF : (EuclideanSpace ℝ (Fin n) × ℝ) →L[ℝ] ℝ :=
    fderiv ℝ F ((0 : EuclideanSpace ℝ (Fin n)), (0 : ℝ))

  -- The "time-direction" derivative must vanish because δL 0 = 0.
  have h_time : dF ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ)) = 0 := by
    have h0 :
        δL (0 : EuclideanSpace ℝ (Fin n)) =
          fderiv ℝ F ((0 : EuclideanSpace ℝ (Fin n)), (0 : ℝ))
            ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ)) := by
      simpa using (hEq (0 : EuclideanSpace ℝ (Fin n))
        (0 : EuclideanSpace ℝ (Fin n)) (0 : ℝ))
    have : dF ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ)) =
        δL (0 : EuclideanSpace ℝ (Fin n)) := by
      simpa [dF] using h0.symm
    simpa [hδL0] using this

  -- Induced continuous linear functional on velocity: v ↦ dF (v,0).
  let φ : (EuclideanSpace ℝ (Fin n)) →L[ℝ] ℝ :=
    dF.comp (ContinuousLinearMap.inl ℝ (EuclideanSpace ℝ (Fin n)) ℝ)

  -- Show δL v = φ v for all v.
  have hφ : ∀ v : EuclideanSpace ℝ (Fin n), δL v = φ v := by
    intro v
    have hv :
        δL v =
          fderiv ℝ F ((0 : EuclideanSpace ℝ (Fin n)), (0 : ℝ))
            (v, (1 : ℝ)) := by
      simpa using (hEq (0 : EuclideanSpace ℝ (Fin n)) v (0 : ℝ))
    have hv' : δL v = dF (v, (1 : ℝ)) := by
```

```
      simpa [dF] using hv
    calc
      δL v = dF (v, (1 : ℝ)) := hv'
      _ = dF ((v, (0 : ℝ)) + ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ))) :=
        simp
      _ = dF (v, (0 : ℝ)) + dF ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ)) :=
        simpa using
          (dF.map_add (v, (0 : ℝ)) ((0 : EuclideanSpace ℝ (Fin n)), (1 : ℝ)
      _ = dF (v, (0 : ℝ)) := by
        simp [h_time]
      _ = φ v := by
        simp [φ]

  -- Frechet–Riesz: represent φ as inner product with some g.
  refine ⟨(InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin n))).symm φ, ?
  intro v
  have hinner :
      ⟪(InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin n))).symm φ, v⟫_ℝ
    rw [InnerProductSpace.toDual_symm_apply (𝕜 := ℝ)
        (E := EuclideanSpace ℝ (Fin n)) (x := v) (y := φ)]
  calc
    δL v = φ v := hφ v
    _ = ⟪(InnerProductSpace.toDual ℝ (EuclideanSpace ℝ (Fin n))).symm φ, v⟫
      rw [hinner.symm]

end Lagrangian

end ClassicalMechanics
```

1.6 PhysLean/CondensedMatter/TightBindingChain/Basic.lean

```
/-- The adjoint of localizedComp |m⟩⟨n| is |n⟩⟨m|. -/
lemma localizedComp_adjoint (m n : Fin T.N) (ψ φ : T.HilbertSpace) :
    ⟪|m⟩⟨n| ψ, φ⟫_ℂ = ⟪ψ, |n⟩⟨m| φ⟫_ℂ := by
  simp only [localizedComp, LinearMap.coe_mk, AddHom.coe_mk]
  rw [inner_smul_left, inner_smul_right, inner_conj_symm]
  ring

    ⟪T.hamiltonian ψ, φ⟫_ℂ = ⟪ψ, T.hamiltonian φ⟫_ℂ := by
  simp only [hamiltonian, LinearMap.sub_apply, LinearMap.smul_apply, Linear
    Finset.sum_apply, LinearMap.add_apply]
  rw [inner_sub_left, inner_sub_right]
  congr 1
  · -- E0 term
```

```
    simp only [Finset.smul_sum]
    rw [sum_inner, inner_sum]
    apply Finset.sum_congr rfl
    intro n _
    simp only [inner_smul_left_eq_smul, inner_smul_right_eq_smul]
    rw [localizedComp_adjoint]
  · -- t term
    simp only [Finset.smul_sum, smul_add]
    rw [sum_inner, inner_sum]
    apply Finset.sum_congr rfl
    intro n _
    rw [inner_add_left, inner_add_right]
    simp only [inner_smul_left_eq_smul, inner_smul_right_eq_smul]
    rw [localizedComp_adjoint, localizedComp_adjoint]
    ring
/-- The energy eigenstates of the tight binding chain are orthogonal.

This is a fundamental quantum mechanical result: eigenstates of a Hermitian operator
(the Hamiltonian) with distinct eigenvalues are orthogonal. Here we prove it directl
using the periodic boundary conditions which quantize the wavenumbers.

The key physical insight is that different wavenumbers k₁ ≠ k₂ give rise to differen
N-th roots of unity exp(i(k₂-k₁)a), and the sum of all N-
th roots of unity equals zero. -/
    Pairwise fun k1 k2 => ⟪T.energyEigenstate k1, T.energyEigenstate k2⟫_ℂ = 0 := by
  intro k1 k2 hne
  simp only [energyEigenstate, sum_inner]
  simp_rw [inner_sum, inner_smul_left, inner_smul_right,
    orthonormal_iff_ite.mp T.localizedState_orthonormal]
  simp only [mul_ite, mul_one, mul_zero, Finset.sum_ite_eq, Finset.mem_univ, ↓reduce
  set ω := Complex.exp (Complex.I * (k2 - k1) * T.a) with hω_def
  have hsum_eq : ∑ n : Fin T.N, (starRingEnd ℂ) (Complex.exp (Complex.I * k1 * n * T
      Complex.exp (Complex.I * k2 * n * T.a) = ∑ i ∈ Finset.range T.N, ω ^ i := by
    rw [Fin.sum_univ_eq_sum_range (fun n =>
      (starRingEnd ℂ) (Complex.exp (Complex.I * k1 * n * T.a)) *
      Complex.exp (Complex.I * k2 * n * T.a))]
    refine Finset.sum_congr rfl fun i _ => ?_
    rw [starRingEnd_apply, Complex.star_def, ← Complex.exp_conj]
    simp only [map_mul, Complex.conj_I, Complex.conj_ofReal, Complex.conj_natCast]
    rw [← Complex.exp_add, hω_def, ← Complex.exp_nat_mul]
    ring_nf
  rw [hsum_eq]
  have hω_pow : ω ^ T.N = 1 := by
    simp only [hω_def, ← Complex.exp_nat_mul]
    have h2 := quantaWaveNumber_exp_N T 1 k2
    have h1 := quantaWaveNumber_exp_N T 1 k1
```

```
    simp only [Nat.cast_one] at h2 h1
    calc _ = Complex.exp (Complex.I * k2 * 1 * T.N * T.a - Complex.I * k1 *
             ring_nf
         _ = 1 := by rw [Complex.exp_sub, h2, h1, div_one]
  have hω_ne_one : ω ≠ 1 := by
    intro hω_eq_one
    apply hne
    obtain ⟨_, ⟨n1, rfl⟩⟩ := k1
    obtain ⟨_, ⟨n2, rfl⟩⟩ := k2
    simp only [Subtype.mk.injEq]
    have hexp := Complex.exp_eq_one_iff.mp (hω_def ▸ hω_eq_one)
    obtain ⟨m, hm⟩ := hexp
    have ha : (T.a : ℂ) ≠ 0 := Complex.ne_zero_of_re_pos T.a_pos
    have hN : (T.N : ℂ) ≠ 0 := by simp [Ne.symm (NeZero.ne' T.N)]
    simp only [Complex.ofReal_mul, Complex.ofReal_div, Complex.ofReal_ofNat
      Complex.ofReal_natCast, Complex.ofReal_sub] at hm
    field_simp at hm
    have hm_int : (n2 : ℤ) - n1 = T.N * m := by
      have hm_eq : (n2 : ℂ) - n1 = (T.N : ℂ) * m := by ring_nf at hm ⊢; exa
      exact_mod_cast congrArg Complex.re hm_eq
    have hn1_lt : (n1 : ℤ) < T.N := by exact_mod_cast n1.isLt
    have hn2_lt : (n2 : ℤ) < T.N := by exact_mod_cast n2.isLt
    have hN_pos : (0 : ℤ) < T.N := by exact_mod_cast Nat.pos_of_ne_zero (Ne
    have hm_bound : m = 0 := by
      have h1 : -(T.N : ℤ) < (n2 : ℤ) - n1 := by omega
      have h2 : (n2 : ℤ) - n1 < T.N := by omega
      rw [hm_int] at h1 h2
      nlinarith
    simp only [hm_bound, mul_zero] at hm_int
    have heq : n1.val = n2.val := by omega
    simp only [heq]
  -- Use the geometric series formula: (ω - 1) * ∑ω^i = ω^N - 1
  -- Since ω^N = 1 and ω ≠ 1, the sum must be zero
  have hgeom := mul_geom_sum ω T.N
  rw [hω_pow, sub_self] at hgeom
  exact mul_eq_zero.mp hgeom |>.resolve_left (sub_ne_zero.mpr hω_ne_one)
```

1.7 PhysLean/Electromagnetism/Dynamics/IsExtrema.lean

```
\frac{1}{\epsilon_0} \left(\frac{\partial J_i}{\partial x_j} -
```

1.8 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean

```
import Mathlib.Analysis.Matrix.Normed
import Mathlib.Analysis.Matrix.Order
## i. Overview

The two Higgs doublet model (2HDM) is an extension of the Standard Model which adds
doublet.

## References

- https://arxiv.org/abs/hep-ph/0605184
- https://arxiv.org/abs/1605.03237

/-- The configuration space of the two Higgs doublet model.
  In otherwords, the underlying vector space associated with the model. -
/

namespace TwoHiggsDoublet

open InnerProductSpace

@[ext]
lemma ext_of_fst_snd {H1 H2 : TwoHiggsDoublet}
    (h1 : H1.Φ1 = H2.Φ1) (h2 : H1.Φ2 = H2.Φ2) : H1 = H2 := by
  cases H1
  cases H2
  congr
/-!

## B. Gauge group actions

-/

noncomputable instance : SMul StandardModel.GaugeGroupI TwoHiggsDoublet where
  smul g H :=
    { Φ1 := g • H.Φ1
      Φ2 := g • H.Φ2 }

@[simp]
lemma gaugeGroupI_smul_fst (g : StandardModel.GaugeGroupI) (H : TwoHiggsDoublet) :
    (g • H).Φ1 = g • H.Φ1 := rfl

@[simp]
lemma gaugeGroupI_smul_snd (g : StandardModel.GaugeGroupI) (H : TwoHiggsDoublet) :
    (g • H).Φ2 = g • H.Φ2 := rfl
```

```
noncomputable instance : MulAction StandardModel.GaugeGroupI TwoHiggsDouble
  one_smul H := by
    ext <;> simp
  mul_smul g1 g2 H := by
    ext <;> simp [mul_smul]

end TwoHiggsDoublet
```

1.9 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/GramMatrix

```
/-
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joseph Tooby-Smith
-/
import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Basic
/-!

# The gram matrix for the two Higgs doublet model

The main reference for material in this section is https://arxiv.org/pdf/he
ph/0605184.

We will show that the gram matrix of the two Higgs doublet model
describes the gauge orbits of the configuration space.

-/
namespace TwoHiggsDoublet

open InnerProductSpace
open StandardModel

/-!

## A. The Gram matrix

-/

/-- The Gram matrix of the two Higgs doublet.
  This matrix is used in https://arxiv.org/abs/hep-ph/0605184. -
/
noncomputable def gramMatrix (H : TwoHiggsDoublet) : Matrix (Fin 2) (Fin 2)
  !![‖H.Φ1‖ ^ 2, ⟪H.Φ1, H.Φ2⟫_ℂ; ⟪H.Φ2, H.Φ1⟫_ℂ, ‖H.Φ2‖ ^ 2]
```

```
lemma gramMatrix_selfAdjoint (H : TwoHiggsDoublet) :
    IsSelfAdjoint (gramMatrix H) := by
  rw [gramMatrix]
  ext i j
  fin_cases i <;> fin_cases j <;> simp [inner_conj_symm]

lemma eq_fst_norm_of_eq_gramMatrix {H1 H2 : TwoHiggsDoublet}
    (h : H1.gramMatrix = H2.gramMatrix) : ‖H1.Φ1‖ = ‖H2.Φ1‖ := by
  rw [gramMatrix, gramMatrix] at h
  have h1 := congrArg (fun x => x 0 0) h
  simp only [Matrix.of_apply, Matrix.cons_val', Matrix.cons_val_zero, Fin.isValue] a
  rw [inner_self_eq_norm_sq_to_K, inner_self_eq_norm_sq_to_K] at h1
  rw [sq_eq_sq_iff_eq_or_eq_neg] at h1
  rcases h1 with h1 | h1
  · simpa using h1
  · rw [← RCLike.ofReal_neg] at h1
    have hnorm1 : 0 ≤ ‖H1.Φ1‖ := norm_nonneg H1.Φ1
    have hnorm2 : 0 ≤ ‖H2.Φ1‖ := norm_nonneg H2.Φ1
    have hl : ‖H1.Φ1‖ = (-‖H2.Φ1‖) := Eq.symm
      ((fun {z w} => Complex.ofReal_inj.mp) (id (Eq.symm h1)))
    grind

lemma eq_snd_norm_of_eq_gramMatrix {H1 H2 : TwoHiggsDoublet}
    (h : H1.gramMatrix = H2.gramMatrix) : ‖H1.Φ2‖ = ‖H2.Φ2‖ := by
  rw [gramMatrix, gramMatrix] at h
  have h1 := congrArg (fun x => x 1 1) h
  simp [Matrix.of_apply, Matrix.cons_val', Matrix.cons_val_one, Fin.isValue] at h1
  rw [sq_eq_sq_iff_eq_or_eq_neg] at h1
  rcases h1 with h1 | h1
  · simpa using h1
  · erw [← RCLike.ofReal_neg] at h1
    have hnorm1 : 0 ≤ ‖H1.Φ2‖ := norm_nonneg H1.Φ2
    have hnorm2 : 0 ≤ ‖H2.Φ2‖ := norm_nonneg H2.Φ2
    have hl : ‖H1.Φ2‖ = (-‖H2.Φ2‖) := Eq.symm
      ((fun {z w} => Complex.ofReal_inj.mp) (id (Eq.symm h1)))
    grind

@[simp]
lemma gaugeGroupI_smul_gramMatrix (g : StandardModel.GaugeGroupI) (H : TwoHiggsDoubl
    (g • H).gramMatrix = H.gramMatrix := by
  rw [gramMatrix, gramMatrix, gaugeGroupI_smul_fst, gaugeGroupI_smul_snd]
  ext i j
  fin_cases i <;> fin_cases j <;> simp

lemma gramMatrix_det_eq (H : TwoHiggsDoublet) :
```

```
    H.gramMatrix.det = ‖H.Φ1‖ ^ 2 * ‖H.Φ2‖ ^ 2 - ‖⟪H.Φ1, H.Φ2⟫_ℂ‖ ^ 2 := by
  rw [gramMatrix, Matrix.det_fin_two]
  simp only [inner_self_eq_norm_sq_to_K, Complex.coe_algebraMap, Fin.isValu
    Matrix.cons_val', Matrix.cons_val_zero, Matrix.cons_val_fin_one, Matrix
    sub_right_inj]
  rw [← Complex.conj_mul']
  simp only [inner_conj_symm]

lemma gramMatrix_det_eq_real (H : TwoHiggsDoublet) :
    H.gramMatrix.det.re = ‖H.Φ1‖ ^ 2 * ‖H.Φ2‖ ^ 2 - ‖⟪H.Φ1, H.Φ2⟫_ℂ‖ ^ 2 :=
  rw [gramMatrix_det_eq]
  simp [← Complex.ofReal_pow, Complex.ofReal_im]

lemma gramMatrix_det_nonneg (H : TwoHiggsDoublet) :
    0 ≤ H.gramMatrix.det.re := by
  rw [gramMatrix_det_eq_real]
  simp only [sub_nonneg]
  convert inner_mul_inner_self_le (𝕜 := ℂ) H.Φ1 H.Φ2
  · simp [sq, norm_inner_symm]
  · exact norm_sq_eq_re_inner H.Φ1
  · exact norm_sq_eq_re_inner H.Φ2

lemma gramMatrix_tr_nonneg (H : TwoHiggsDoublet) :
    0 ≤ H.gramMatrix.trace.re := by
  rw [gramMatrix, Matrix.trace_fin_two]
  simp only [inner_self_eq_norm_sq_to_K, Complex.coe_algebraMap, Fin.isValu
    Matrix.cons_val', Matrix.cons_val_zero, Matrix.cons_val_fin_one, Matrix
    Complex.add_re]
  apply add_nonneg
  · rw [← Complex.ofReal_pow, Complex.ofReal_re]
    exact sq_nonneg ‖H.Φ1‖
  · rw [← Complex.ofReal_pow, Complex.ofReal_re]
    exact sq_nonneg ‖H.Φ2‖

lemma gaugeGroupI_exists_fst_eq {H : TwoHiggsDoublet} (h1 : H.Φ1 ≠ 0) :
    ∃ g : StandardModel.GaugeGroupI,
      g • H.Φ1 = (!₂[‖H.Φ1‖, 0] : HiggsVec) ∧
      (g • H.Φ2) 0 = ⟪H.Φ1, H.Φ2⟫_ℂ / ‖H.Φ1‖ ∧
      ‖(g • H.Φ2) 1‖ = Real.sqrt (H.gramMatrix.det.re) / ‖H.Φ1‖ := by
  rw [gramMatrix_det_eq_real]
  obtain ⟨g, h⟩ := (HiggsVec.mem_orbit_gaugeGroupI_iff (H.Φ1) (!₂[‖H.Φ1‖, 0
    (by simp [@PiLp.norm_eq_of_L2])
  use g
  simp at h
  simp [h]
  have h_fst : (g • H.Φ2).ofLp 0 = ⟪H.Φ1, H.Φ2⟫_ℂ / ‖H.Φ1‖ := by
```

```
    have h2 : ⟪H.Φ1, H.Φ2⟫_ℂ = ⟪g • H.Φ1, g • H.Φ2⟫_ℂ := by
      simp
    rw [h] at h2
    conv_rhs at h2 =>
      simp [PiLp.inner_apply]
    rw [h2]
    have hx : (‖H.Φ1‖ : ℂ) ≠ 0 := by
      simp_all
    field_simp
  apply And.intro h_fst
  have hx : ‖g • H.Φ2‖ ^ 2 = ‖H.Φ2‖ ^ 2 := by
    simp
  rw [PiLp.norm_sq_eq_of_L2] at hx
  simp at hx
  have hx0 : ‖(g • H.Φ2).ofLp 1‖ ^ 2 = ‖H.Φ2‖ ^ 2 - ‖(g • H.Φ2).ofLp 0‖ ^ 2 := by
    rw [← hx]
    simp
  have h0 : ‖(g • H.Φ2) 1‖ ^ 2 = (‖H.Φ1‖ ^ 2 * ‖H.Φ2‖ ^ 2 - ‖⟪H.Φ1, H.Φ2⟫_ℂ‖ ^ 2) /
    field_simp
    rw [hx0, h_fst]
    simp only [Fin.isValue, Complex.norm_div, Complex.norm_real, norm_norm]
    ring_nf
    field_simp
  have habc (a b c : ℝ) (ha : 0 ≤ a) (hx : a ^ 2 = b / c ^2) (hc : c ≠ 0) (hc : 0 <
      a = Real.sqrt b / c := by
    field_simp
    symm
    have hb : b = a ^ 2 * c ^ 2 := by
      rw [hx]
      field_simp
    subst hb
    rw [Real.sqrt_eq_iff_eq_sq]
    · ring
    · positivity
    · positivity
  apply habc
  rw [h0]
  ring_nf
  · exact norm_ne_zero_iff.mpr h1
  · simpa using h1
  · exact norm_nonneg ((g • H.Φ2).ofLp 1)

lemma gaugeGroupI_exists_fst_eq_snd_eq {H : TwoHiggsDoublet} (h1 : H.Φ1 ≠ 0) :
    ∃ g : StandardModel.GaugeGroupI,
      g • H.Φ1 = (!₂[‖H.Φ1‖, 0] : HiggsVec) ∧
      g • H.Φ2 = (!₂[⟪H.Φ1, H.Φ2⟫_ℂ / ‖H.Φ1‖, √(H.gramMatrix.det.re) / ‖H.Φ1‖] : Hig
```

```
  obtain ⟨g, h_fst, h_snd_0, h_snd_1⟩ := gaugeGroupI_exists_fst_eq h1
  obtain ⟨k, h1, h2, h3⟩ := HiggsVec.gaugeGroupI_smul_phase_snd (g • H.Φ2)
  use k * g
  apply And.intro
  · rw [mul_smul, h_fst, h3]
  · rw [mul_smul]
    ext i
    fin_cases i
    · simp
      rw [h2, h_snd_0]
    · simp
      rw [h1, h_snd_1]
      simp

lemma mem_orbit_gaugeGroupI_iff_gramMatrix (H1 H2 : TwoHiggsDoublet) :
    H1 ∈ MulAction.orbit GaugeGroupI H2 ↔ H1.gramMatrix = H2.gramMatrix :=
  apply Iff.intro
  · intro h
    obtain ⟨g, hg⟩ := h
    simp at hg
    simp [← hg]
  by_cases Φ1_zero : H1.Φ1 = 0
  · intro h
    obtain ⟨g1, hg1⟩ := (HiggsVec.mem_orbit_gaugeGroupI_iff (H1.Φ2) (!₂[‖H1
      (by simp [@PiLp.norm_eq_of_L2])
    obtain ⟨g2, hg2⟩ := (HiggsVec.mem_orbit_gaugeGroupI_iff (H2.Φ2) (!₂[‖H2
      (by simp [@PiLp.norm_eq_of_L2])
    use g1⁻¹ * g2
    simp only
    ext:1
    · simp [Φ1_zero]
      have hnorm : ‖H2.Φ1‖ = ‖H1.Φ1‖ := by
        symm
        rw [← eq_fst_norm_of_eq_gramMatrix h]
      simp [Φ1_zero] at hnorm
      simp [hnorm]
    · simp [mul_smul]
      refine inv_smul_eq_iff.mpr ?_
      simp at hg1 hg2
      simp [hg1, hg2]
      exact eq_snd_norm_of_eq_gramMatrix (id (Eq.symm h))
  · intro h
    obtain ⟨g1, H1_Φ1, H1_Φ2⟩ := gaugeGroupI_exists_fst_eq_snd_eq (H := H1)
    have Φ2_nezero : H2.Φ1 ≠ 0 := by
      intro hzero
      have hnorm : ‖H1.Φ1‖ = ‖H2.Φ1‖ := by
```

```
        rw [← eq_fst_norm_of_eq_gramMatrix h]
      simp [hzero] at hnorm
      simp [hnorm] at Φ1_zero
    obtain ⟨g2, H2_Φ1, H2_Φ2⟩ := gaugeGroupI_exists_fst_eq_snd_eq (H := H2) Φ2_nezer
    use g1⁻¹ * g2
    simp only
    ext:1
    · simp [mul_smul]
      refine inv_smul_eq_iff.mpr ?_
      simp [H1_Φ1, H2_Φ1]
      apply eq_fst_norm_of_eq_gramMatrix (id (Eq.symm h))
    · simp [mul_smul]
      refine inv_smul_eq_iff.mpr ?_
      simp [H1_Φ2, H2_Φ2]
      apply And.intro
      · congr 1
        · symm
          exact congrArg (fun x => x 1 0) h
        · simp only [Complex.ofReal_inj]
          exact eq_fst_norm_of_eq_gramMatrix (id (Eq.symm h))
      · congr 2
        · simp [h]
        · exact eq_fst_norm_of_eq_gramMatrix (id (Eq.symm h))

/-!

### A.1. Gram matrix is surjective

-/

open ComplexConjugate

lemma gramMatrix_surjective_det_tr (K : Matrix (Fin 2) (Fin 2) ℂ)
    (hKs : IsSelfAdjoint K) (hKdet : 0 ≤ K.det.re) (hKtr : 0 ≤ K.trace.re) :
    ∃ H : TwoHiggsDoublet, H.gramMatrix = K := by
  /- Basic results related to K. -/
  have hK_explicit : K = !![K 0 0, K 0 1; K 1 0, K 1 1] := by
    ext i j
    fin_cases i <;> fin_cases j <;> simp
  have hK_star_explicit : star K = !![star (K 0 0), star (K 1 0); star (K 0 1), star
    ext i j
    fin_cases i <;> fin_cases j <;> simp
  rw [isSelfAdjoint_iff, hK_star_explicit] at hKs
  conv_rhs at hKs => rw [hK_explicit]
  simp at hKs
  have hK_explicit2 : K = !![((K 0 0).re : ℂ), K 0 1; conj (K 0 1), ((K 1 1).re : ℂ)
```

```
    conv_lhs => rw [hK_explicit]
    simp [hKs]
    apply And.intro
    · refine Eq.symm ((fun {z} => Complex.conj_eq_iff_re.mp) ?_)
      simp [hKs]
    · refine Eq.symm ((fun {z} => Complex.conj_eq_iff_re.mp) ?_)
      simp [hKs]
  clear hK_explicit hK_star_explicit hKs
  generalize (K 0 0).re = a at *
  generalize (K 1 1).re = b at *
  generalize K 0 1 = c at *
  have det_eq_abc : K.det = a * b - ‖c‖ ^ 2 := by
    simp [hK_explicit2]
    rw [Complex.mul_conj']
  have tra_eq_abc : K.trace.re = a + b := by
    simp [hK_explicit2]
  simp [det_eq_abc, ← Complex.ofReal_pow] at hKdet
  rw [tra_eq_abc] at hKtr
  rw [hK_explicit2]
  clear hK_explicit2 det_eq_abc tra_eq_abc
  have ha_nonneg : 0 ≤ a := by nlinarith
  have hb_nonneg : 0 ≤ b := by nlinarith
  /- Splitting the cases into a = 0 and other. -/
  by_cases ha : a = 0
  · use ⟨(0 : HiggsVec), (!₂[√b, 0] : HiggsVec)⟩
    subst ha
    simp_all
    subst hKdet
    ext i j
    fin_cases i <;> fin_cases j <;> simp [gramMatrix]
    simp [PiLp.norm_eq_of_L2, ← Complex.ofReal_pow]
    exact Real.sq_sqrt hb_nonneg
  /- The case when a ≠ 0. -/
  have h1 : (√a : ℂ) ≠ 0 := by
      simp_all
  use ⟨(!₂[√a, 0] : HiggsVec), !₂[conj c/ √a, √(a * b - ‖c‖ ^ 2) / √a]⟩
  ext i j
  fin_cases i <;> fin_cases j <;> simp [gramMatrix, PiLp.norm_eq_of_L2, ← C
  · exact Real.sq_sqrt ha_nonneg
  · simp [PiLp.inner_apply]
    field_simp
  · simp [PiLp.inner_apply]
    field_simp
  · rw [Real.sq_sqrt, abs_of_nonneg, abs_of_nonneg]
    field_simp
    rw [Real.sq_sqrt, Real.sq_sqrt]
```

```
    ring
    · positivity
    · nlinarith
    · exact Real.sqrt_nonneg (a * b - ‖c‖ ^ 2)
    · positivity
    · positivity

/-!

## B. The Gram vector

-/

/-- A real vector containing the components of the Gram matrix in the Pauli basis. -
/
noncomputable def gramVector (H : TwoHiggsDoublet) : Fin 1 ⊕ Fin 3 → ℝ := fun μ =>
  2 * PauliMatrix.pauliBasis.repr ⟨gramMatrix H, gramMatrix_selfAdjoint H⟩ μ

@[simp]
lemma gaugeGroupI_smul_fst_gramVector (g : StandardModel.GaugeGroupI)
    (H : TwoHiggsDoublet) (μ : Fin 1 ⊕ Fin 3) :
    (g • H).gramVector μ = H.gramVector μ := by
  rw [gramVector, gramVector]
  congr 1
  simp

lemma gramMatrix_eq_gramVector_sum_pauliMatrix (H : TwoHiggsDoublet) :
    gramMatrix H = (1 / 2 : ℝ) • ∑ μ, H.gramVector μ • PauliMatrix.pauliMatrix μ :=
  have h1 := congrArg (fun x => x.1) <|
    PauliMatrix.pauliBasis.sum_repr ⟨gramMatrix H, gramMatrix_selfAdjoint H⟩
  simp [-Module.Basis.sum_repr] at h1
  rw [← h1]
  simp [gramVector, smul_smul, Finset.smul_sum]
  congr 1
  · simp [PauliMatrix.pauliBasis, PauliMatrix.pauliSelfAdjoint]
  · simp [PauliMatrix.pauliBasis, PauliMatrix.pauliSelfAdjoint]

lemma gramMatrix_eq_component_gramVector (H : TwoHiggsDoublet) :
    gramMatrix H =
    !![(1 / 2 : ℂ) * (H.gramVector (Sum.inl 0) + H.gramVector (Sum.inr 2)),
      (1 / 2 : ℂ) * (H.gramVector (Sum.inr 0) - Complex.I * H.gramVector (Sum.inr 1)
      (1 / 2 : ℂ) * (H.gramVector (Sum.inr 0) + Complex.I * H.gramVector (Sum.inr 1)
      (1 / 2 : ℂ) * (H.gramVector (Sum.inl 0) - H.gramVector (Sum.inr 2))] := by
  rw [gramMatrix_eq_gramVector_sum_pauliMatrix]
  simp only [one_div, PauliMatrix.pauliMatrix, Matrix.one_fin_two, Fintype.sum_sum_t
    Finset.univ_unique, Fin.default_eq_zero, Fin.isValue, Finset.sum_singleton, Matr
```

```
    Matrix.smul_cons, Complex.real_smul, mul_one, smul_zero, Matrix.smul_em
    smul_neg, Matrix.of_add_of, Matrix.add_cons, Matrix.head_cons, add_zero
    Matrix.empty_add_empty, zero_add, smul_add, Complex.ofReal_inv, Complex
    EmbeddingLike.apply_eq_iff_eq, Matrix.vecCons_inj, and_true]
  ring_nf
  simp

lemma gramVector_inl_eq_trace_gramMatrix (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inl 0) = H.gramMatrix.trace.re := by
  rw [gramMatrix_eq_component_gramVector, Matrix.trace_fin_two]
  simp only [Fin.isValue, one_div, Matrix.of_apply, Matrix.cons_val', Matri
    Matrix.cons_val_fin_one, Matrix.cons_val_one]
  ring_nf
  simp

lemma gramVector_inl_nonneg (H : TwoHiggsDoublet) :
    0 ≤ H.gramVector (Sum.inl 0) := by
  rw [gramVector_inl_eq_trace_gramMatrix]
  exact gramMatrix_tr_nonneg H

lemma normSq_Φ1_eq_gramVector (H : TwoHiggsDoublet) :
    ‖H.Φ1‖ ^ 2 = (1/2 : ℝ) * (H.gramVector (Sum.inl 0) + H.gramVector (Sum.
  trans (gramMatrix H 0 0).re
  · simp [gramMatrix]
    rw [← Complex.ofReal_pow, Complex.ofReal_re]
  · rw [gramMatrix_eq_component_gramVector]
    simp

lemma normSq_Φ2_eq_gramVector (H : TwoHiggsDoublet) :
    ‖H.Φ2‖ ^ 2 = (1/2 : ℝ) * (H.gramVector (Sum.inl 0) - H.gramVector (Sum.
  trans (gramMatrix H 1 1).re
  · simp [gramMatrix]
    rw [← Complex.ofReal_pow, Complex.ofReal_re]
  · rw [gramMatrix_eq_component_gramVector]
    simp

lemma Φ1_inner_Φ2_eq_gramVector (H : TwoHiggsDoublet) :
    ⟪H.Φ1, H.Φ2⟫_ℂ = (1/2 : ℝ) * (H.gramVector (Sum.inr 0) +
    Complex.I * H.gramVector (Sum.inr 1)) := by
  trans (gramMatrix H 1 0)
  · simp [gramMatrix]
  · simp [gramMatrix_eq_component_gramVector]

lemma Φ2_inner_Φ1_eq_gramVector (H : TwoHiggsDoublet) :
    ⟪H.Φ2, H.Φ1⟫_ℂ = (1/2 : ℝ) * (H.gramVector (Sum.inr 0) -
    Complex.I * H.gramVector (Sum.inr 1)) := by
```

```
  trans (gramMatrix H 0 1)
  · simp [gramMatrix]
  · simp [gramMatrix_eq_component_gramVector]

open ComplexConjugate

lemma Φ1_inner_Φ2_normSq_eq_gramVector (H : TwoHiggsDoublet) :
    ‖⟪H.Φ1, H.Φ2⟫_ℂ‖ ^ 2 =
    (1/4 : ℝ) * (H.gramVector (Sum.inr 0) ^ 2 + H.gramVector (Sum.inr 1) ^ 2) := by
  trans (⟪H.Φ1, H.Φ2⟫_ℂ * conj ⟪H.Φ1, H.Φ2⟫_ℂ).re
  · rw [Complex.mul_conj', ← Complex.ofReal_pow]
    rfl
  rw [conj_inner_symm H.Φ2 H.Φ1]
  rw [Φ1_inner_Φ2_eq_gramVector, Φ2_inner_Φ1_eq_gramVector]
  simp only [one_div, Complex.ofReal_inv, Complex.ofReal_ofNat, Fin.isValue, Complex
    Complex.inv_re, Complex.re_ofNat, Complex.normSq_ofNat, div_self_mul_self', Comp
    Complex.ofReal_re, Complex.I_re, zero_mul, Complex.I_im, Complex.ofReal_im, mul_
    add_zero, Complex.inv_im, Complex.im_ofNat, neg_zero, zero_div, Complex.add_im,
    one_mul, zero_add, sub_zero, Complex.sub_re, Complex.sub_im, zero_sub, mul_neg,
  ring

lemma gramVector_inl_zero_eq (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inl 0) = ‖H.Φ1‖ ^ 2 + ‖H.Φ2‖ ^ 2 := by
  rw [normSq_Φ1_eq_gramVector, normSq_Φ2_eq_gramVector]
  ring

lemma gramVector_inl_zero_eq_gramMatrix (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inl 0) = (H.gramMatrix 0 0).re + (H.gramMatrix 1 1).re := by
  simp [gramVector_inl_zero_eq, gramMatrix, ← Complex.ofReal_pow, Complex.ofReal_re]

lemma gramVector_inr_zero_eq (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inr 0) = 2 * (⟪H.Φ1, H.Φ2⟫_ℂ).re := by
  rw [Φ1_inner_Φ2_eq_gramVector]
  simp

lemma gramVector_inr_zero_eq_gramMatrix (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inr 0) = 2 * (H.gramMatrix 1 0).re := by
  rw [gramMatrix, gramVector_inr_zero_eq]
  simp

lemma gramVector_inr_one_eq (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inr 1) = 2 * (⟪H.Φ1, H.Φ2⟫_ℂ).im := by
  rw [Φ1_inner_Φ2_eq_gramVector]
  simp

lemma gramVector_inr_one_eq_gramMatrix (H : TwoHiggsDoublet) :
```

```
    H.gramVector (Sum.inr 1) = 2 * (H.gramMatrix 1 0).im := by
  rw [gramMatrix, gramVector_inr_one_eq]
  simp

lemma gramVector_inr_two_eq (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inr 2) = ‖H.Φ1‖ ^ 2 - ‖H.Φ2‖ ^ 2 := by
  rw [normSq_Φ1_eq_gramVector, normSq_Φ2_eq_gramVector]
  ring

lemma gramVector_inr_two_eq_gramMatrix (H : TwoHiggsDoublet) :
    H.gramVector (Sum.inr 2) = (H.gramMatrix 0 0).re - (H.gramMatrix 1 1).r
  simp [gramVector_inr_two_eq, gramMatrix, ← Complex.ofReal_pow, Complex.of

lemma gramMatrix_det_eq_gramVector (H : TwoHiggsDoublet) :
    H.gramMatrix.det.re =
    (1/4 : ℝ) * (H.gramVector (Sum.inl 0) ^ 2 -
    ∑ μ : Fin 3, H.gramVector (Sum.inr μ) ^ 2) := by
  rw [gramMatrix_det_eq_real]
  simp [normSq_Φ1_eq_gramVector, normSq_Φ2_eq_gramVector, Φ1_inner_Φ2_normS
    Fin.sum_univ_three]
  ring

lemma gramVector_inr_sum_sq_le_inl (H : TwoHiggsDoublet) :
    ∑ μ : Fin 3, H.gramVector (Sum.inr μ) ^ 2 ≤ H.gramVector (Sum.inl 0) ^
  apply sub_nonneg.mp
  trans (4 : ℝ) * H.gramMatrix.det.re
  · apply mul_nonneg
    · norm_num
    · exact gramMatrix_det_nonneg H
  apply (le_of_eq _)
  rw [gramMatrix_det_eq_gramVector]
  ring

lemma gramVector_surjective (v : Fin 1 ⊕ Fin 3 → ℝ)
    (h_inl : 0 ≤ v (Sum.inl 0))
    (h_det : ∑ μ : Fin 3, v (Sum.inr μ) ^ 2 ≤ v (Sum.inl 0) ^ 2) :
    ∃ H : TwoHiggsDoublet, H.gramVector = v := by
  let K := !![(1 / 2 : ℂ) * (v (Sum.inl 0) + v (Sum.inr 2)),
      (1 / 2 : ℂ) * (v (Sum.inr 0) - Complex.I * v (Sum.inr 1));
      (1 / 2 : ℂ) * (v (Sum.inr 0) + Complex.I * v (Sum.inr 1)),
      (1 / 2 : ℂ) * (v (Sum.inl 0) - v (Sum.inr 2))]
  have K_star : star K = !![(1 / 2 : ℂ) * (v (Sum.inl 0) + v (Sum.inr 2)),
      (1 / 2 : ℂ) * (v (Sum.inr 0) - Complex.I * v (Sum.inr 1));
      (1 / 2 : ℂ) * (v (Sum.inr 0) + Complex.I * v (Sum.inr 1)),
      (1 / 2 : ℂ) * (v (Sum.inl 0) - v (Sum.inr 2))] := by
    ext i j
```

```
    fin_cases i <;> fin_cases j <;> simp [K]
    ring
  have hK_selfAdjoint : IsSelfAdjoint K := by
    exact K_star
  have hK_det_nonneg : 0 ≤ K.det.re := by
    simp [K]
    simp [Fin.sum_univ_three] at h_det
    linarith
  have hK_tr : 0 ≤ K.trace.re := by
    simp [K]
    linarith
  obtain ⟨H, hH⟩ := gramMatrix_surjective_det_tr K hK_selfAdjoint hK_det_nonneg hK_t
  use H
  ext μ
  fin_cases μ
  · simp [gramVector_inl_zero_eq_gramMatrix, hH, K]
    ring
  · simp [gramVector_inr_zero_eq_gramMatrix, hH, K]
  · simp [gramVector_inr_one_eq_gramMatrix, hH, K]
  · simp [gramVector_inr_two_eq_gramMatrix, hH, K]
    ring

end TwoHiggsDoublet
```

1.10 PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Potential.lean

```
Copyright (c) 2025 Joseph Tooby-Smith. All rights reserved.
import PhysLean.Particles.BeyondTheStandardModel.TwoHDM.GramMatrix
namespace TwoHiggsDoublet
open InnerProductSpace
/-- The parameters of the Two Higgs doublet model potential.
  Following the convention of https://arxiv.org/pdf/1605.03237. -
/
structure PotentialParameters where
namespace PotentialParameters

/-- A reparameterization of the parameters of the quadratic terms of the
  potential for use with the gramVector. -/
noncomputable def ξ (P : PotentialParameters) : Fin 1 ⊕ Fin 3 → ℝ := fun μ =>
  match μ with
  | Sum.inl 0 => (P.m₁₁2 + P.m₂₂2) / 2
  | Sum.inr 0 => -Complex.re P.m₁₂2
  | Sum.inr 1 => Complex.im P.m₁₂2
  | Sum.inr 2 => (P.m₁₁2 - P.m₂₂2) / 2
```

```
/-- A reparameterization of the parameters of the quartic terms of the
  potential for use with the gramVector. -/
noncomputable def η (P : PotentialParameters) : Fin 1 ⊕ Fin 3 → Fin 1 ⊕ Fin
  | Sum.inl 0, Sum.inl 0 => (P.l₁ + P.l₂ + 2 * P.l₃) / 8
  | Sum.inl 0, Sum.inr 0 => (P.l₆.re + P.l₇.re) * (1 / 4)
  | Sum.inl 0, Sum.inr 1 => (P.l₆.im + P.l₇.im) * (-1 / 4)
  | Sum.inl 0, Sum.inr 2 => (P.l₁ - P.l₂) * (1 / 8)
  | Sum.inr 0, Sum.inl 0 => (P.l₆.re + P.l₇.re) * (1 / 4)
  | Sum.inr 1, Sum.inl 0 => (P.l₆.im + P.l₇.im) * (-1 / 4)
  | Sum.inr 2, Sum.inl 0 => (P.l₁ - P.l₂) * (1 / 8)
  /-η_a_a-/
  | Sum.inr 0, Sum.inr 0 => (P.l₅.re + P.l₄) * (1 / 4)
  | Sum.inr 1, Sum.inr 1 => (-P.l₅.re + P.l₄) * (1 / 4)
  | Sum.inr 2, Sum.inr 2 => (P.l₁ + P.l₂ - 2 * P.l₃) * (1 / 8)
  | Sum.inr 0, Sum.inr 1 => P.l₅.im * (-1 / 4)
  | Sum.inr 2, Sum.inr 0 => (P.l₆.re - P.l₇.re) * (1 / 4)
  | Sum.inr 2, Sum.inr 1 => (P.l₇.im - P.l₆.im) * (1 / 4)
  | Sum.inr 1, Sum.inr 0 => P.l₅.im * (-1 / 4)
  | Sum.inr 0, Sum.inr 2 => (P.l₆.re - P.l₇.re) * (1 / 4)
  | Sum.inr 1, Sum.inr 2 => (P.l₇.im - P.l₆.im) * (1 / 4)

lemma η_symm (P : PotentialParameters) (μ ν : Fin 1 ⊕ Fin 3) :
    P.η μ ν = P.η ν μ := by
  fin_cases μ <;> fin_cases ν <;> simp [η]

end PotentialParameters
open ComplexConjugate
/-- The mass term of the two Higgs doublet model potential. -
/
noncomputable def massTerm (P : PotentialParameters) (H : TwoHiggsDoublet)
  P.m₁₁2 * ‖H.Φ1‖ ^ 2 + P.m₂₂2 * ‖H.Φ2‖ ^ 2 -
  (P.m₁₂2 * ⟪H.Φ1, H.Φ2⟫_ℂ + conj P.m₁₂2 * ⟪H.Φ2, H.Φ1⟫_ℂ).re

lemma massTerm_eq_gramVector (P : PotentialParameters) (H : TwoHiggsDoublet
    massTerm P H = ∑ μ, P.ξ μ * H.gramVector μ := by
  simp [massTerm, Fin.sum_univ_three, PotentialParameters.ξ, normSq_Φ1_eq_g
    normSq_Φ2_eq_gramVector, Φ1_inner_Φ2_eq_gramVector, Φ2_inner_Φ1_eq_gram
  ring

@[simp]
lemma gaugeGroupI_smul_massTerm (g : StandardModel.GaugeGroupI) (P : Potent
    (H : TwoHiggsDoublet) :
    massTerm P (g • H) = massTerm P H := by
  rw [massTerm_eq_gramVector, massTerm_eq_gramVector]
/-- The quartic term of the two Higgs doublet model potential. -
```

```
/
noncomputable def quarticTerm (P : PotentialParameters) (H : TwoHiggsDoublet) : ℝ :=
  1/2 * P.l₁ * ‖H.Φ1‖ ^ 2 * ‖H.Φ1‖ ^ 2 + 1/2 * P.l₂ * ‖H.Φ2‖ ^ 2 * ‖H.Φ2‖ ^ 2
  + P.l₃ * ‖H.Φ1‖ ^ 2 * ‖H.Φ2‖ ^ 2
  + P.l₄ * ‖⟪H.Φ1, H.Φ2⟫_ℂ‖ ^ 2
  + (1/2 * P.l₅ * ⟪H.Φ1, H.Φ2⟫_ℂ ^ 2 + 1/2 * conj P.l₅ * ⟪H.Φ2, H.Φ1⟫_ℂ ^ 2).re
  + (P.l₆ * ‖H.Φ1‖ ^ 2 * ⟪H.Φ1, H.Φ2⟫_ℂ + conj P.l₆ * ‖H.Φ1‖ ^ 2 * ⟪H.Φ2, H.Φ1⟫_ℂ).r
  + (P.l₇ * ‖H.Φ2‖ ^ 2 * ⟪H.Φ1, H.Φ2⟫_ℂ + conj P.l₇ * ‖H.Φ2‖ ^ 2 * ⟪H.Φ2, H.Φ1⟫_ℂ).r

lemma quarticTerm_l₄_expand (P : PotentialParameters) (H : TwoHiggsDoublet) :
    H.quarticTerm P =
    1/2 * P.l₁ * ‖H.Φ1‖ ^ 2 * ‖H.Φ1‖ ^ 2 + 1/2 * P.l₂ * ‖H.Φ2‖ ^ 2 * ‖H.Φ2‖ ^ 2
    + P.l₃ * ‖H.Φ1‖ ^ 2 * ‖H.Φ2‖ ^ 2
    + P.l₄ * (⟪H.Φ1, H.Φ2⟫_ℂ * ⟪H.Φ2, H.Φ1⟫_ℂ).re
    + (1/2 * P.l₅ * ⟪H.Φ1, H.Φ2⟫_ℂ ^ 2 + 1/2 * conj P.l₅ * ⟪H.Φ2, H.Φ1⟫_ℂ ^ 2).re
    + (P.l₆ * ‖H.Φ1‖ ^ 2 * ⟪H.Φ1, H.Φ2⟫_ℂ + conj P.l₆ * ‖H.Φ1‖ ^ 2 * ⟪H.Φ2, H.Φ1⟫_ℂ)
    + (P.l₇ * ‖H.Φ2‖ ^ 2 * ⟪H.Φ1, H.Φ2⟫_ℂ + conj P.l₇ * ‖H.Φ2‖ ^ 2 * ⟪H.Φ2, H.Φ1⟫_ℂ)
  simp [quarticTerm]
  left
  rw [Complex.sq_norm]
  rw [← Complex.mul_re]
  rw [← inner_conj_symm, ← Complex.normSq_eq_conj_mul_self]
  simp only [inner_conj_symm, Complex.ofReal_re]
  rw [← inner_conj_symm]
  exact Complex.normSq_conj ⟪H.Φ2, H.Φ1⟫_ℂ

lemma quarticTerm_eq_gramVector (P : PotentialParameters) (H : TwoHiggsDoublet) :
    quarticTerm P H = ∑ a, ∑ b, H.gramVector a * H.gramVector b * P.η a b := by
  simp [quarticTerm_l₄_expand, Fin.sum_univ_three, PotentialParameters.η, normSq_Φ1_
    normSq_Φ2_eq_gramVector, Φ1_inner_Φ2_eq_gramVector, Φ2_inner_Φ1_eq_gramVector]
  simp [← Complex.ofReal_pow, Complex.ofReal_re, normSq_Φ1_eq_gramVector,
    normSq_Φ2_eq_gramVector]
  ring

@[simp]
lemma gaugeGroupI_smul_quarticTerm (g : StandardModel.GaugeGroupI) (P : PotentialPar
    (H : TwoHiggsDoublet) :
    quarticTerm P (g • H) = quarticTerm P H := by
  rw [quarticTerm_eq_gramVector, quarticTerm_eq_gramVector]
  simp

/-- The potential of the two Higgs doublet model. -/
noncomputable def potential (P : PotentialParameters) (H : TwoHiggsDoublet) : ℝ :=
  massTerm P H + quarticTerm P H

@[simp]
```

```
lemma gaugeGroupI_smul_potential (g : StandardModel.GaugeGroupI)
    (P : PotentialParameters) (H : TwoHiggsDoublet) :
    potential P (g • H) = potential P H := by
  rw [potential, potential]
  simp
/-!

## Boundedness of the potential

-/

/-- The condition that the potential is bounded from below. -
/
def PotentialIsBounded (P : PotentialParameters) : Prop :=
  ∃ c : ℝ, ∀ H : TwoHiggsDoublet, c ≤ potential P H

lemma potentialIsBounded_iff_forall_gramVector (P : PotentialParameters) :
    PotentialIsBounded P ↔ ∃ c : ℝ, ∀ K : Fin 1 ⊕ Fin 3 → ℝ, 0 ≤ K (Sum.inl
      ∑ μ : Fin 3, K (Sum.inr μ) ^ 2 ≤ K (Sum.inl 0) ^ 2 →
      c ≤ ∑ μ, P.ξ μ * K μ + ∑ a, ∑ b, K a * K b * P.η a b := by
  apply Iff.intro
  · intro h
    obtain ⟨c, hc⟩ := h
    use c
    intro v hv₀ hv_sum
    obtain ⟨H, hH⟩ := gramVector_surjective v hv₀ hv_sum
    apply (hc H).trans
    apply le_of_eq
    rw [potential, massTerm_eq_gramVector, quarticTerm_eq_gramVector]
    simp [hH]
  · intro h
    obtain ⟨c, hc⟩ := h
    use c
    intro H
    apply (hc H.gramVector (gramVector_inl_nonneg H) (gramVector_inr_sum_sq
    apply le_of_eq
    rw [potential, massTerm_eq_gramVector, quarticTerm_eq_gramVector]

lemma potentialIsBounded_iff_forall_euclid (P : PotentialParameters) :
    PotentialIsBounded P ↔ ∃ c, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0
      ‖K‖ ^ 2 ≤ K0 ^ 2 → c ≤ P.ξ (Sum.inl 0) * K0 + ∑ μ, P.ξ (Sum.inr μ) *
      + K0 ^ 2 * P.η (Sum.inl 0) (Sum.inl 0)
      + 2 * K0 * ∑ b, K b * P.η (Sum.inl 0) (Sum.inr b) +
      ∑ a, ∑ b, K a * K b * P.η (Sum.inr a) (Sum.inr b) := by
  rw [potentialIsBounded_iff_forall_gramVector]
  refine exists_congr (fun c => ?_)
```

```
  rw [Equiv.forall_congr_left (Equiv.sumArrowEquivProdArrow (Fin 1) (Fin 3) ℝ)]
  simp only [Fin.isValue, Fintype.sum_sum_type, Finset.univ_unique, Fin.default_eq_z
    Finset.sum_singleton, Prod.forall, Equiv.sumArrowEquivProdArrow_symm_apply_inl,
    Equiv.sumArrowEquivProdArrow_symm_apply_inr]
  rw [Equiv.forall_congr_left <| Equiv.funUnique (Fin 1) ℝ]
  apply forall_congr'
  intro K0
  rw [Equiv.forall_congr_left <| (WithLp.equiv 2 ((i : Fin 3) → (fun x => ℝ) i)).sym
  apply forall_congr'
  intro K
  simp only [Fin.isValue, Equiv.funUnique_symm_apply, uniqueElim_const, Equiv.symm_s
    WithLp.equiv_apply]
  refine imp_congr_right ?_
  intro hle
  simp only [PiLp.norm_sq_eq_of_L2]
  simp only [Fin.isValue, Real.norm_eq_abs, sq_abs]
  refine imp_congr_right ?_
  intro hle'
  apply le_iff_le_of_cmp_eq_cmp
  congr 1
  simp [add_assoc, sq, Finset.sum_add_distrib]
  simp [mul_assoc, ← Finset.mul_sum]
  conv_lhs =>
    enter [2, 2, 2, i]
    rw [PotentialParameters.η_symm]
lemma potentialIsBounded_iff_forall_euclid_lt (P : PotentialParameters) :
    PotentialIsBounded P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ K : EuclideanSpace ℝ (Fin 3), 0 < K0
      ‖K‖ ^ 2 ≤ K0 ^ 2 → c ≤ P.ξ (Sum.inl 0) * K0 + ∑ μ, P.ξ (Sum.inr μ) * K μ
      + K0 ^ 2 * P.η (Sum.inl 0) (Sum.inl 0)
      + 2 * K0 * ∑ b, K b * P.η (Sum.inl 0) (Sum.inr b) +
      ∑ a, ∑ b, K a * K b * P.η (Sum.inr a) (Sum.inr b) := by
  rw [potentialIsBounded_iff_forall_euclid]
  apply Iff.intro
  · intro h
    obtain ⟨c, hc⟩ := h
    use c
    apply And.intro
    · simpa using hc 0 0 (by simp) (by simp)
    · intro K0 K hk0 hle
      exact hc K0 K hk0.le hle
  · intro h
    obtain ⟨c, hc₀, hc⟩ := h
    use c
    intro K0 K hK0 hle
    by_cases hK0' : K0 = 0
    · subst hK0'
```

```
      simp_all
    · refine hc K0 K ?_ hle
      grind

/-!

## Mass term reduced

-/

/-- A function related to the mass term of the potential, used in the bound
  condition and equivalent to the term `J2` in
  https://arxiv.org/abs/hep-ph/0605184. -/
noncomputable def massTermReduced (P : PotentialParameters) (k : EuclideanS
  P.ξ (Sum.inl 0) + ∑ μ, P.ξ (Sum.inr μ) * k μ

lemma massTermReduced_lower_bound (P : PotentialParameters) (k : EuclideanS
    (hk : ‖k‖ ^ 2 ≤ 1) : P.ξ (Sum.inl 0) - √(∑ a, |P.ξ (Sum.inr a)| ^ 2) ≤
  simp only [Fin.isValue, massTermReduced]
  have h1 (a b c : ℝ) (h : - b ≤ c) : a - b ≤ a + c:= by grind
  apply h1
  let ξEuclid : EuclideanSpace ℝ (Fin 3) := WithLp.toLp 2 (fun a => P.ξ (Su
  trans - ‖ξEuclid‖
  · simp [@PiLp.norm_eq_of_L2, ξEuclid]
  trans - (‖k‖ * ‖ξEuclid‖)
  · simp
    simp at hk
    have ha (a b : ℝ) (h : a ≤ 1) (ha : 0 ≤ a) (hb : 0 ≤ b) : a * b ≤ b :=
    apply ha
    · exact hk
    · exact norm_nonneg k
    · exact norm_nonneg ξEuclid
  trans - ‖⟪k, ξEuclid⟫_ℝ‖
  · simp
    exact abs_real_inner_le_norm k ξEuclid
  trans ⟪k, ξEuclid⟫_ℝ
  · simp
    grind
  simp [PiLp.inner_apply, ξEuclid]
## Quartic term reduced
/-- A function related to the quartic term of the potential, used in the bo
  condition and equivalent to the term `J4` in
  https://arxiv.org/abs/hep-ph/0605184. -/
noncomputable def quarticTermReduced (P : PotentialParameters) (k : Euclide
  P.η (Sum.inl 0) (Sum.inl 0) + 2 * ∑ b, k b * P.η (Sum.inl 0) (Sum.inr b)
  ∑ a, ∑ b, k a * k b * P.η (Sum.inr a) (Sum.inr b)
```

```
lemma potentialIsBounded_iff_exists_forall_forall_reduced (P : PotentialParameters)
    PotentialIsBounded P ↔ ∃ c ≤ 0, ∀ K0 : ℝ, ∀ k : EuclideanSpace ℝ (Fin 3), 0 < K0
      ‖k‖ ^ 2 ≤ 1 → c ≤ K0 * massTermReduced P k + K0 ^ 2 * quarticTermReduced P k :
  rw [potentialIsBounded_iff_forall_euclid_lt]
  refine exists_congr <| fun c => and_congr_right <| fun hc => forall_congr' <| fun
  apply Iff.intro
  · refine fun h k hK0 k_le_one => (h (K0 • k) hK0 ?_).trans (le_of_eq ?_)
    · simp [norm_smul]
      rw [abs_of_nonneg (by positivity), mul_pow]
      nlinarith
    · simp [add_assoc, massTermReduced, quarticTermReduced]
      ring_nf
      simp [add_assoc, mul_assoc, ← Finset.mul_sum, sq]
      ring
  · intro h K hK0 hle
    refine (h ((1 / K0) • K) hK0 ?_).trans (le_of_eq ?_)
    · simp [norm_smul]
      field_simp
      rw [sq_le_sq] at hle
      simpa using hle
    · simp [add_assoc, massTermReduced, quarticTermReduced]
      ring_nf
      simp [add_assoc, mul_assoc, ← Finset.mul_sum, sq]
      field_simp
      ring_nf
      simp only [← Finset.sum_mul, Fin.isValue]
      field_simp
      ring

lemma quarticTermReduced_nonneg_of_potentialIsBounded (P : PotentialParameters)
    (hP : PotentialIsBounded P) (k : EuclideanSpace ℝ (Fin 3))
    (hk : ‖k‖ ^ 2 ≤ 1) : 0 ≤ quarticTermReduced P k := by
  rw [potentialIsBounded_iff_exists_forall_forall_reduced] at hP
  suffices hp : ∀ (a b : ℝ), (∃ c ≤ 0, ∀ x, 0 < x → c ≤ a * x + b * x ^ 2) →
      0 ≤ b ∧ (b = 0 → 0 ≤ a) by
    obtain ⟨c, hc, h⟩ := hP
    refine (hp (massTermReduced P k) (quarticTermReduced P k) ⟨c, hc, ?_⟩).1
    grind
  intro a b
  by_cases hb : b = 0
  /- The case of b = 0. -/
  · subst hb
    by_cases ha : a = 0
    · subst ha
      simp
```

```
    · simp only [zero_mul, add_zero, le_refl, forall_const, true_and]
      rintro ⟨c, hc, hx⟩
      by_contra h2
      simp_all
      refine not_lt_of_ge (hx (c/a + 1) ?_) ?_
      · exact add_pos_of_nonneg_of_pos (div_nonneg_of_nonpos hc (Std.le_of_
          Real.zero_lt_one
      · field_simp
        grind
  /- The case of b ≠ 0. -/
  have h1 (x : ℝ) : a * x + b * x ^ 2 = b * (x + a / (2 * b)) ^ 2 - a ^ 2 /
  generalize a ^ 2 / (4 * b) = c1 at h1
  generalize a / (2 * b) = d at h1
  simp only [hb, IsEmpty.forall_iff, and_true]
  have hlt (c : ℝ) (x : ℝ) : (c ≤ a * x + b * x ^ 2) ↔ c + c1 ≤ b * (x + d)
  conv_lhs => enter [1, c, 2, x]; rw [hlt c]
  trans ∃ c, ∀ x, 0 < x → c ≤ b * (x + d) ^ 2
  · rintro ⟨c, hc, hx⟩
    use c + c1
  rintro ⟨c, hc⟩
  by_contra hn
  suffices hs : ∀ x, x ^ 2 ≤ c/b from not_lt_of_ge (hs √(|c/b| + 1)) (by gr
  suffices hs : ∀ x, 0 < x → (x + d) ^ 2 ≤ c/b from
    fun x => le_trans ((Real.sqrt_le_left (by grind)).mp
      (by grind [Real.sqrt_sq_eq_abs])) (hs (|x| + |d| + 1) (by positivity)
  exact fun x hx => (le_div_iff_of_neg (by grind)).mpr (by grind)

lemma potentialIsBounded_iff_exists_forall_reduced (P : PotentialParameters
    PotentialIsBounded P ↔ ∃ c, 0 ≤ c ∧ ∀ k : EuclideanSpace ℝ (Fin 3), ‖k‖
      0 ≤ quarticTermReduced P k ∧
      (massTermReduced P k < 0 →
      massTermReduced P k ^ 2 ≤ 4 * quarticTermReduced P k * c) := by
  rw [potentialIsBounded_iff_exists_forall_forall_reduced]
  refine Iff.intro (fun ⟨c, hc, h⟩ => ⟨-c, by grind, fun k hk => ?_⟩)
    (fun ⟨c, hc, h⟩ => ⟨-c, by grind, fun K0 k hk0 hk => ?_⟩)
  · have hJ4_nonneg : 0 ≤ quarticTermReduced P k := by
      refine quarticTermReduced_nonneg_of_potentialIsBounded P ?_ k hk
      rw [potentialIsBounded_iff_exists_forall_forall_reduced]
      exact ⟨c, hc, h⟩
    have h0 : ∀ K0, 0 < K0 → c ≤ K0 * massTermReduced P k + K0 ^ 2 * quarti
      fun K0 a => h K0 k a hk
    clear h
    generalize massTermReduced P k = j2 at *
    generalize quarticTermReduced P k = j4 at *
    by_cases j4_zero : j4 = 0
    · subst j4_zero
```

```
      simp_all
      intro hj2
      by_contra hn
      specialize h0 ((c - 1) / j2) <| by
        refine div_pos_iff.mpr (Or.inr ?_)
        grind
      field_simp at h0
      linarith
    · have hsq (K0 : ℝ) : K0 * j2 + K0 ^ 2 * j4 =
            j4 * (K0 + j2 / (2 * j4)) ^ 2 - j2 ^ 2 / (4 * j4) := by
        grind
      have hj_pos : 0 < j4 := by grind
      apply And.intro
      · grind
      · intro j2_neg
        conv at h0 => enter [2]; rw [hsq]
        specialize h0 (- j2 / (2 * j4)) <| by
          field_simp
          grind
        ring_nf at h0
        field_simp at h0
        grind
  · specialize h k hk
    generalize massTermReduced P k = j2 at *
    generalize quarticTermReduced P k = j4 at *
    by_cases hJ4 : j4 = 0
    · subst j4
      simp_all
      trans 0
      · grind
      · by_cases hJ2 : j2 = 0
        · simp_all
        · simp_all
    · have hJ4_pos : 0 < j4 := by grind
      have h0 : K0 * j2 + K0 ^ 2 * j4 = j4 * (K0 + j2 / (2 * j4)) ^ 2 - j2 ^ 2 / (4
          grind
      rw [h0]
      by_cases hJ2_neg : j2 < 0
      · trans j4 * (K0 + j2 / (2 * j4)) ^ 2 - c
        · nlinarith
        · field_simp
          grind
      · refine neg_le_sub_iff_le_add.mpr ?_
        trans j4 * (K0 + j2 / (2 * j4)) ^ 2
        · nlinarith
        · grind
```

```
lemma massTermReduced_pos_of_quarticTermReduced_zero_potentialIsBounded (P
    (hP : PotentialIsBounded P) (k : EuclideanSpace ℝ (Fin 3))
    (hk : ‖k‖ ^ 2 ≤ 1) (hq : quarticTermReduced P k = 0) : 0 ≤ massTermRedu
  rw [potentialIsBounded_iff_exists_forall_reduced] at hP
  obtain ⟨c, hc₀, hc⟩ := hP
  specialize hc k hk
  rw [hq] at hc
  simp only [le_refl, mul_zero, zero_mul, sq_nonpos_iff, true_and] at hc
  generalize massTermReduced P k = j2 at *
  grind

@[sorryful]
lemma potentialIsBounded_iff_forall_reduced (P : PotentialParameters) :
    PotentialIsBounded P ↔ ∀ k : EuclideanSpace ℝ (Fin 3), ‖k‖ ^ 2 ≤ 1 →
      0 ≤ quarticTermReduced P k ∧ (quarticTermReduced P k = 0 → 0 ≤ massTe
  sorry

end TwoHiggsDoublet
```

1.11 PhysLean/Particles/StandardModel/Basic.lean

```
/-- The inclusion of a U(1) subgroup. -/
def ofU1Subgroup (u1 : unitary ℂ) : GaugeGroupI :=
  (1,
  ⟨!![star (u1 ^ 3 : unitary ℂ), 0;0, (u1 ^ 3 : unitary ℂ)], by
    simp only [SetLike.mem_coe]
    rw [mem_unitaryGroup_iff']
    funext i j
    rw [Matrix.mul_apply]
    fin_cases i <;> fin_cases j <;> simp [conj_mul'], by
    simp only [RCLike.star_def, SetLike.mem_coe, MonoidHom.mem_mker, coe_de
      det_fin_two_of, conj_mul', mul_zero, sub_zero]
    simp⟩, u1⟩

@[simp]
lemma ofU1Subgroup_toSU3 (u1 : unitary ℂ) :
    toSU3 (ofU1Subgroup u1) = 1 := rfl

@[simp]
lemma ofU1Subgroup_toSU2 (u1 : unitary ℂ) :
    toSU2 (ofU1Subgroup u1) = ⟨!![star (u1 ^ 3 : unitary ℂ), 0;0, (u1 ^ 3 :
    simp only [SetLike.mem_coe]
    rw [mem_unitaryGroup_iff']
```

```
    funext i j
    rw [Matrix.mul_apply]
    fin_cases i <;> fin_cases j <;> simp [conj_mul'], by
    simp only [RCLike.star_def, SetLike.mem_coe, MonoidHom.mem_mker, coe_detMonoidHo
      det_fin_two_of, conj_mul', mul_zero, sub_zero]
    simp⟩ := rfl

@[simp]
lemma ofU1Subgroup_toU1 (u1 : unitary ℂ) :
    toU1 (ofU1Subgroup u1) = u1 := rfl
```

1.12 PhysLean/Particles/StandardModel/HiggsBoson/Basic.lean

```
lemma gaugeGroupI_smul_eq_U1_smul_SU2 (g : StandardModel.GaugeGroupI) (φ : HiggsVec)
    g • φ = (WithLp.toLp 2 <| (g.toU1 ^ 3 • g.toSU2.1) *ᵥ φ.ofLp) := by
  rw [gaugeGroupI_smul_eq]
  rw [Matrix.smul_mulVec]

instance : DistribMulAction StandardModel.GaugeGroupI HiggsVec where
  smul_zero g := by
    rw [gaugeGroupI_smul_eq_U1_smul_SU2]
    simp
  smul_add g φ ψ := by
    rw [gaugeGroupI_smul_eq_U1_smul_SU2]
    simp [mulVec_add]
    simp [← gaugeGroupI_smul_eq_U1_smul_SU2]
/-!

## A.8. Gauge action removing phase from second component

-/

lemma ofU1Subgroup_smul_eq_smul (g : unitary ℂ) (φ : HiggsVec) :
    (StandardModel.GaugeGroupI.ofU1Subgroup g) • φ =
    (WithLp.toLp 2 <| !![1, 0; 0, g.1 ^ 6] *ᵥ φ.ofLp) := by
  rw [gaugeGroupI_smul_eq_U1_smul_SU2]
  simp only [GaugeGroupI.ofU1Subgroup_toU1, GaugeGroupI.ofU1Subgroup_toSU2, Submonoi
    star_pow, RCLike.star_def, smul_of, smul_cons, smul_zero, smul_empty, cons_mulVe
    cons_dotProduct, zero_mul, dotProduct_of_isEmpty, add_zero, zero_add, empty_mulV
    WithLp.toLp.injEq, vecCons_inj, mul_eq_mul_right_iff, and_true]
  apply And.intro
  · have h0 : g ^ 3 • (starRingEnd ℂ) ↑g ^ 3 = 1 := by
      trans (normSq (g ^ 3).1 : ℂ)
      · rw [← mul_conj]
```

```
        simp
        rfl
      · rw [normSq_eq_norm_sq]
        simp
    simp [h0]
  · left
    trans (g ^ 3 : ℂ) • (g ^ 3 : ℂ)
    · rfl
    simp only [smul_eq_mul]
    ring

lemma gaugeGroupI_smul_phase_snd (φ : HiggsVec) :
    ∃ g : StandardModel.GaugeGroupI,
      (g • φ).ofLp 1 = ‖(φ.ofLp 1)‖ ∧
      (∀ φ1 : HiggsVec, (g • φ1).ofLp 0 = φ1.ofLp 0) ∧
      (∀ a : ℝ, g • (!₂[a, 0] : HiggsVec) = (!₂[a, 0] : HiggsVec)) := by
  let θ := arg (φ 1)
  use StandardModel.GaugeGroupI.ofU1Subgroup ⟨Complex.exp (-
I * θ / 6), by
    rw [Unitary.mem_iff]
    simp [← Complex.exp_conj, ← Complex.exp_add, Complex.conj_ofNat]
    ring_nf
    simp⟩
  apply And.intro
  · rw [ofU1Subgroup_smul_eq_smul]
    simp only [Fin.isValue, neg_mul, cons_mulVec, cons_dotProduct, one_mul,
      dotProduct_of_isEmpty, add_zero, zero_add, empty_mulVec, cons_val_one
    trans Complex.exp (-I * θ / 6) ^ 6 * φ.ofLp 1
    · congr
      simp
    have habs : φ.ofLp 1 = cexp (I * arg (φ.ofLp 1)) * ‖φ.ofLp 1‖ := by
      conv_lhs => rw [← Complex.norm_mul_exp_arg_mul_I (φ.ofLp 1)]
      ring_nf
    conv_lhs => rw [habs]
    rw [← mul_assoc, ← Complex.exp_nat_mul, ← Complex.exp_add]
    simp [θ]
    ring_nf
    simp
  apply And.intro
  · intro φ
    rw [ofU1Subgroup_smul_eq_smul]
    simp
    rfl
  · intro a
    simp [ofU1Subgroup_smul_eq_smul]
    ext i
```

```
    fin_cases i <;> simp
```

1.13 PhysLean/Particles/SuperSymmetry/SU5/ChargeSpectrum/Completions.l

```
/-- If `x` is a subset of `y` and `y` is complete, then there is a completion of `x`
  a subset of `y`. -/
```

1.14 PhysLean/QFT/AnomalyCancellation/Basic.lean

```
  intro S T h
  ext
  simpa [ACCSystemQuad.quadSolsInclLinSols] using
    congrArg (fun X => X.val) h
  intro S T h
  have h' : χ.quadSolsInclLinSols S = χ.quadSolsInclLinSols T := by
    apply ACCSystemLinear.linSolsIncl_injective (χ := χ.toACCSystemLinear)
    simpa [ACCSystemQuad.quadSolsIncl, MulActionHom.comp_apply] using h
  exact quadSolsInclLinSols_injective χ h'
/--
The type of charges plus the anomaly cancellation conditions.

In many physical settings these conditions are derived formally from the gauge group
fermionic representations. They arise from triangle Feynman diagrams, and can also b
using index-theoretic or characteristic-class constructions.

In this file, we take the resulting conditions as input data: linear, quadratic and
homogeneous forms on the space of rational charges.
-/
  intro S T h
  apply Sols.ext
  have hv : (χ.solsInclQuadSols S).val = (χ.solsInclQuadSols T).val :=
    congrArg (fun X => X.val) h
  simpa [ACCSystem.solsInclQuadSols] using hv
  intro S T h
  have h' : χ.solsInclQuadSols S = χ.solsInclQuadSols T := by
    apply ACCSystemQuad.quadSolsInclLinSols_injective (χ := χ.toACCSystemQuad)
    simpa [ACCSystem.solsInclLinSols, MulActionHom.comp_apply] using h
  exact solsInclQuadSols_injective χ h'
  intro S T h
  have h' : χ.solsInclQuadSols S = χ.solsInclQuadSols T := by
    apply ACCSystemQuad.quadSolsIncl_injective (χ := χ.toACCSystemQuad)
    simpa [ACCSystem.solsIncl, MulActionHom.comp_apply] using h
```

```
  exact (solsInclQuadSols_injective χ) h'
One natural direction is to formalize how the anomaly cancellation conditio
`ACCSystem` arise from gauge-theoretic data (a gauge group together with fe
Physically these arise from triangle Feynman diagrams, and can also be desc
theoretic
or characteristic-class constructions (e.g. through an anomaly polynomial).
formalize this derivation in Lean, and instead take the resulting homogeneo
```

1.15 PhysLean/QuantumMechanics/OneDimension/ReflectionlessPoter

```
    ((-Complex.I * Q.ℏ * Q.κ) / Real.sqrt (2 * Q.m)) • Q.tanhOperatorSchwar
```

1.16 PhysLean/QuantumMechanics/PlanckConstant.lean

```
/-- The value of the reduced Planck's constant in units of J.s. -
/
def ℏ : Subtype fun x : ℝ => 0 < x := ⟨1.054571817e-34, by norm_num⟩
```

1.17 PhysLean/Relativity/LorentzAlgebra/Basis.lean

```
This file defines the 6 standard generators of the Lorentz algebra so(1,3)
- J_0 (rotation about x-axis) : Acts on (y,z) components
- J_1 (rotation about y-axis) : Acts on (z,x) components
- J_2 (rotation about z-axis) : Acts on (x,y) components
            else if μ = Sum.inr 2 ∧ ν = Sum.inr 1 then 1
            else 0
      | 1 => if μ = Sum.inr 0 ∧ ν = Sum.inr 2 then 1
            else if μ = Sum.inr 2 ∧ ν = Sum.inr 0 then -1
            else 0
      | 2 => if μ = Sum.inr 0 ∧ ν = Sum.inr 1 then -1
            else if μ = Sum.inr 1 ∧ ν = Sum.inr 0 then 1
            else 0
```

1.18 PhysLean/Relativity/Tensors/Basic.lean

```
@[simp, nolint simpVarHead]
```

1.19 PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean

```
/-- The `PermCond` condition is preserved under `colorToComplex`. -
/
@[simp] lemma permCond_colorToComplex {n m : ℕ}
    {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color}
    {σ : Fin m → Fin n} (h : PermCond c c1 σ) :
    PermCond (colorToComplex ∘ c) (colorToComplex ∘ c1) σ := by
  refine And.intro h.1 ?_
  intro i
  simpa [Function.comp_apply] using congrArg colorToComplex (h.2 i)

@[sorryful]
lemma permT_toComplex {n m : ℕ}
    {c : Fin n → realLorentzTensor.Color}
    {c1 : Fin m → realLorentzTensor.Color}
    {σ : Fin m → Fin n} (h : PermCond c c1 σ) (t : ℝT(3, c)) :
    toComplex (permT (S := realLorentzTensor) σ h t)
      =
    permT (S := complexLorentzTensor) σ (permCond_colorToComplex (c := c) (c1 := c1)
      (toComplex (c := c) t) := by
  sorry

/-- `colorToComplex` commutes with `Fin.append` (as functions). -
/
@[simp]
lemma colorToComplex_append {n m : ℕ}
    (c : Fin n → realLorentzTensor.Color) (c1 : Fin m → realLorentzTensor.Color) :
    (colorToComplex ∘ Fin.append c c1) = Fin.append (colorToComplex ∘ c) (colorToCom
  funext x
  -- breaking down x : Fin (n+m) into left/right parts
  refine Fin.addCases (fun i => ?_) (fun j => ?_) x
  · -- left case: x = castAdd m i
    -- here `simp` should expand `Fin.append` on castAdd
    simp [Fin.append, Function.comp_apply]
  · -- right case: x = natAdd n j
    simp [Fin.append, Function.comp_apply]

/-- `prodT` on the complex side, with colors written as `colorToComplex ∘ Fin.append
This is `prodT` followed by a cast using `colorToComplex_append`. -
/
noncomputable def prodTColorToComplex {n m : ℕ}
    {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color} :
    ℂT(colorToComplex ∘ c) → ℂT(colorToComplex ∘ c1) → ℂT(colorToComplex ∘ Fin.appen
    fun x y => permT (S := complexLorentzTensor) id (by simp) <|
      prodT (S := complexLorentzTensor) x y
```

```
@[sorryful]
lemma prodT_toComplex {n m : ℕ}
    {c : Fin n → realLorentzTensor.Color}
    {c1 : Fin m → realLorentzTensor.Color}
    (t : ℝT(3, c)) (t1 : ℝT(3, c1)) :
    toComplex (c := Fin.append c c1) (prodT (S := realLorentzTensor) t t1)
      =
    prodTColorToComplex (c := c) (c1 := c1)
      (toComplex (c := c) t) (toComplex (c := c1) t1) := by
  sorry

/-- `τ` commutes with `colorToComplex` on the Lorentz `up/down` colors. -
/
@[simp]
lemma tau_colorToComplex (x : realLorentzTensor.Color) :
    (complexLorentzTensor).τ (colorToComplex x) = colorToComplex ((realLore
  cases x <;> rfl
@[sorryful]
lemma contrT_toComplex {n : ℕ}
    {c : Fin (n + 1 + 1) → realLorentzTensor.Color} {i j : Fin (n + 1 + 1)}
    (h : i ≠ j ∧ (realLorentzTensor).τ (c i) = c j) (t : ℝT(3, c)) :
    toComplex (c := c ∘ Pure.dropPairEmb i j) (contrT (S := realLorentzTens
      =
    contrT (S := complexLorentzTensor) n i j (by
        -- если у simp достаточно информации, то это закрывается:
        simpa [Function.comp_apply] using
          And.intro h.1 (by
            -- τ-совместимость
            simpa [tau_colorToComplex] using congrArg colorToComplex h.2
          )
      )
      (toComplex (c := c) t) := by
  sorry

@[simp]
lemma complex_repDim_up :
    (complexLorentzTensor).repDim complexLorentzTensor.Color.up = 4 := rfl

@[simp]
lemma complex_repDim_down :
    (complexLorentzTensor).repDim complexLorentzTensor.Color.down = 4 := rf

/-- Convert an evaluation index from the real repDim to the complex repDim.
/
def evalIdxToComplex {n : ℕ}
    {c : Fin (n + 1) → realLorentzTensor.Color} (i : Fin (n + 1))
```

```
    (b : Fin ((realLorentzTensor).repDim (c i))) :
    Fin ((complexLorentzTensor).repDim ((colorToComplex ∘ c) i)) :=
  Fin.cast (by
    cases hci : c i with
    | up =>
        simp [hci, colorToComplex, Function.comp_apply, complex_repDim_up]
    | down =>
        simp [hci, colorToComplex, Function.comp_apply, complex_repDim_down]
  ) b

/-- `evalT` on the complex side, but with output colors as `colorToComplex ∘ (c ∘ i.
Implemented via `permT (σ := id) (by simp)` as a transport. -
/
noncomputable def evalTColorToComplex {n : ℕ}
    {c : Fin (n + 1) → realLorentzTensor.Color} (i : Fin (n + 1))
    (b : Fin ((realLorentzTensor).repDim (c i))) :
    ℂT(colorToComplex ∘ c) → ℂT(colorToComplex ∘ (c ∘ i.succAbove)) :=
  fun t =>
    permT (S := complexLorentzTensor) (σ := (id : Fin n → Fin n))
      (by
        -- transport ((colorToComplex ∘ c) ∘ i.succAbove) and (colorToComplex ∘ (c ∘
        simp [Function.comp_apply])
      ((TensorSpecies.Tensor.evalT (S := complexLorentzTensor) (c := (colorToComplex
          i (evalIdxToComplex (c := c) i b)) t)

/-- The map `toComplex` commutes with `evalT`. -/
@[sorryful]
lemma evalT_toComplex {n : ℕ}
    {c : Fin (n + 1) → realLorentzTensor.Color}
    (i : Fin (n + 1)) (b : Fin ((realLorentzTensor).repDim (c i))) (t : ℝT(3, c)) :
    toComplex (c := c ∘ i.succAbove)
        ((TensorSpecies.Tensor.evalT (S := realLorentzTensor) (c := c) i b) t)
      =
    evalTColorToComplex (c := c) i b (toComplex (c := c) t) := by
  sorry
```

1.20 PhysLean/StatisticalMechanics/BoltzmannConstant.lean

```
In this module give the value of the Boltzmann constant.
/-- The Boltzmann constant in units of `m ^ 2 kg s ^ (-2) K ^ (-
1)`.
  As long as one does not use the underlying value of this quantity,
  then it can be used as Boltzmann's constant in an arbitrary set of units. -
/
```

```
def kBAx : {p : ℝ | 0 < p} := ⟨1.380649e-23, by norm_num⟩
```

1.21 README.md

```
[![](https://img.shields.io/badge/View_The-Stats-blue)](https://physlean.co
[![api_docs](https://img.shields.io/badge/doc-API_docs-blue)](https://physl
## Requirements of the project
 The project shall contain results (definitions, theorems, lemmas and calc
 The project shall be **organized** by **physics**.

 Each definition in the project shall carry a physics-based **documentatio

 Each module (file) in the project shall carry a physics-
based **documentation**.

 The project shall contain Physics Lean **tactics**, **notation** and **sy

 The project shall *not* be tied to physics axiomizations (e.g. axiomatic

 The content of the project shall be carefully **reviewed** and curated, t

 The project shall be completely open-source, community run and independen

 The project shall not be tied to any specific AI model or tool.

 The project shall be for **main-stream** physics only.
```